



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Analyse de la consommation énergétique du DNS

Rapport de stage ST50 - P2026

LEROY Fernand

FISE-INFO
Data Science et IA

Télécom SudParis

19 Place Marguerite Perey
91120 Palaiseau
www.telecom-sudparis.eu

Tuteur en entreprise
MAROT Michel

Suiveur UTBM
BAALA CANALDA Oumaya

Abstract

Le Domain Name System (DNS) est une infrastructure essentielle d'Internet dont la consommation énergétique reste encore peu étudiée. Ce stage, réalisé dans le cadre du projet Datacenter Energy Consumption Optimisation by DNS EvaluationS (DECODES) à Télécom SudParis, vise à mieux comprendre et modéliser la consommation énergétique d'un résolveur DNS, en se concentrant sur l'implémentation Berkeley Internet Name Domain (Bind9).

Pour atteindre cet objectif, un environnement expérimental a été conçu afin de mesurer la consommation énergétique du résolveur sous différentes configurations (protocoles, utilisation du cache et charge). Les données collectées ont permis d'analyser l'influence de ces paramètres sur la consommation énergétique, d'étudier la complexité algorithmique de Bind9 et de développer deux approches de modélisation : un modèle de machine learning basé sur des métriques système et un modèle analytique fondé sur les réseaux de files d'attente. En parallèle, une analyse de la complexité algorithmique de Bind9 a été réalisée afin de mieux comprendre les facteurs logiciels influençant cette consommation.

Les résultats montrent qu'il est possible de prédire avec précision la consommation énergétique du résolveur à partir des métriques collectées, tout en mettant en évidence les principales composantes responsables de cette consommation. Ces travaux constituent une première étape vers la modélisation énergétique de systèmes distribués plus complexes.

Mots-clés : DNS, Bind9, consommation énergétique, modélisation, machine learning, réseaux de files d'attente, complexité algorithmique.

Remerciements

Je tiens tout d'abord à adresser mes plus sincères remerciements à **Michel Marot**, mon encadrant de stage, pour sa disponibilité, son accompagnement et ses précieux conseils tout au long de ce projet. Son expertise, sa confiance et ses nombreuses discussions scientifiques m'ont permis de progresser tant sur le plan technique que méthodologique et ont grandement contribué à la réussite de ce stage.

Je souhaite également remercier **Rosario Patanè** pour son aide et ses précieux conseils tout au long de ce stage. Son expertise dans le domaine de l'analyse de la complexité algorithmique a été particulièrement précieuse, tout comme son accompagnement lors de la préparation des présentations et de la rédaction de ce rapport. J'adresse également mes remerciements à **Lucas Brehon-Grataloup** pour son accompagnement, sa disponibilité et les échanges constructifs qui ont contribué au bon déroulement de ce stage.

Je tiens également à remercier le **Centre Interdisciplinaire Energy4Climate (E4C)** de l'Institut Polytechnique de Paris (IP Paris), financé en partie dans le cadre du troisième Programme d'investissements d'avenir (ANR-18-EUR-0006-02), pour leur soutien à ces travaux.

Je remercie le projet **DECODES**, soutenu par l'Agence Nationale de la Recherche (ANR-25-CE25-1697), qui a fourni le cadre scientifique dans lequel ce stage a été réalisé.

J'adresse également mes remerciements à l'ensemble des membres du projet **DECODES** pour leur accueil, leur disponibilité et les échanges scientifiques enrichissants qui ont jalonné ces quelques mois. Je remercie également tous les doctorants et stagiaires de Télécom SudParis pour leur bonne humeur, les nombreuses discussions, les échanges d'idées et les moments conviviaux partagés au quotidien, qui ont largement contribué à rendre cette expérience particulièrement agréable.

Je tiens à exprimer toute ma gratitude à mes parents pour leur soutien indéfectible, leurs encouragements et leur confiance tout au long de mes études. Leur accompagnement a été essentiel dans la réalisation de ce parcours et de ce stage.

Enfin, je remercie chaleureusement **Alizée Mesnard**, qui m'a permis de découvrir cette opportunité de stage et sans qui cette expérience n'aurait peut-être pas été possible.

Table des matières

Introduction	9
1 Présentation de Télécom SudParis	10
1.1 Présentation de l'organisme d'accueil	10
1.1.1 Télécom SudParis dans son environnement international, européen et français	10
1.1.2 Télécom SudParis au sein de l'Institut Mines-Télécom et de l'Institut Polytechnique de Paris	10
1.1.3 Le département Réseaux et Services de Télécommunications (RST)	11
1.2 Environnement de travail	11
2 Contexte théorique	13
2.1 Fonctionnement du DNS	13
2.1.1 Serveurs autoritatifs et résolveurs	13
2.1.2 Résolution récursive	14
2.1.3 Le mécanisme de cache	14
2.2 Protocoles utilisés	14
2.2.1 DNS sur User Datagram Protocol (UDP)	14
2.2.2 DNS over TLS (DoT)	15
2.2.3 DNS over HTTPS (DoH)	15
2.3 Présentation de Bind9	15
3 Travail réalisé	16
3.1 Description du projet DECODES	16
3.1.1 Contexte et motivation	17
3.1.2 Objectifs	17
3.1.3 Consortium	18
3.1.4 Organisation du travail	18
3.2 État de l'art	19
3.2.1 Méthodes de mesure de la consommation énergétique	19
3.2.2 Consommation énergétique du DNS	20
3.2.3 Positionnement du projet DECODES	20
3.3 Déroulement du travail	21
3.3.1 Mise en place de la méthodologie de mesure	21
3.3.2 Conception de l'environnement expérimental	26
3.3.3 Implémentation des campagnes de mesures	29
3.4 Analyse des résultats	30
3.4.1 Mesures de consommation énergétique du résolveur DNS	31
3.4.2 Mesures de métriques liées au CPU	35
3.4.3 Mesures de métriques liées au cache de Bind9	35
3.4.4 Étude des requêtes envoyées et reçues par le résolveur DNS	37

3.4.5	Étude de la complexité algorithmique de Bind9	39
3.5	Modélisation	45
3.5.1	Prédiction de la consommation énergétique du DNS	46
3.5.2	Modèle de réseau de files d'attente	49
3.5.3	Méthode d'estimation des paramètres	56
3.5.4	Validation expérimentale	56
3.5.5	Conclusion	59
Conclusion		60
Bibliographie		60
Annexes		62
	Détail des métriques collectées	62
	Types de requêtes DNS	63
	Principaux types de requêtes et codes de réponse DNS	63
	Signification des symboles fréquemment rencontrés dans les traces DNS . .	63

Table des figures

2.1	Exemple simplifié d'une résolution DNS récursive.	13
3.1	Évolution des valeurs associées au cache de Bind9 pour différents ensemble de noms de domaines entre 100 000 et 1 millions de noms de domaines . . .	22
3.2	Évolution des valeurs associées au cache de Bind9 pour différents ensemble de noms de domaines entre 1 et 14 millions de noms de domaines	23
3.3	Consommation énergétique moyenne pour différents ensembles de noms de domaine entre 100,000 et 1 million de noms de domaine	24
3.4	Schéma de l'environnement expérimental	26
3.5	Consommation énergétique moyenne du DNS pour UDP, DNS Over TLS (DoT) et DNS Over HTTPS (DoH) avec et sans utilisation du cache avec des connexions non persistantes pour l'ancien protocole de mesure montrant la consommation du DNS sans résolution récursive	32
3.6	Consommation énergétique moyenne du DNS montrant l'impact du cache et de la persistance des connexions	32
3.7	Consommation énergétique moyenne du DNS montrant avec des connexions persistantes, sans les mesures au niveau du DNS montrant leur impact sur la consommation énergétique	32
3.8	Consommation énergétique moyenne du DNS montrant l'impact du cache et des connexions non persistantes	33
3.9	Consommation énergétique moyenne du DNS montrant avec des connexions non persistantes, sans les mesures au niveau du DNS montrant leur impact sur la consommation énergétique	33
3.10	Consommation énergétique moyenne pour UDP, DoT et DoH avec et sans utilisation du cache avec des connexions non persistantes, sans la surcharge lié à l'outil de mesure <code>rndc</code>	34
3.11	Métriques du CPU les plus pertinentes montrant leur corrélation avec la consommation énergétique du résolveur DNS	35
3.12	Métriques du cache de Bind9 moyennées pour les différents protocoles avec et sans utilisation du cache	36
3.13	Nombre de requêtes envoyées et reçues par le résolveur pour une résolution montrant une décroissance exponentielle avec le Query Per Second (QPS) .	38
3.14	Distribution des types de requêtes pour les résolutions récursives	38
3.15	Distribution des Time To Live (TTL) pour des noms de domaine aléatoires . .	39
3.16	Distribution des TTL pour les Top Level Domain (TLD)	39
3.17	Graphiques de l'analyse de la complexité algorithmique statique de Bind9 . .	40
3.18	Graphe des relations d'appels entre les fonctions de Bind9 avec leur complexités algorithmiques statiques	41
3.19	Graph d'appels des 55 fonctions les plus appelées de Bind9 avec leur complexité algorithmique statique	42

3.20	Nombre d'appels des 10 fonctions les plus appelées de Bind9 en fonction du QPS pour DoH avec utilisation du cache	43
3.21	Nombre d'appels des 40 fonctions les plus appelées de Bind9 avec leur complexité algorithmique statique	44
3.22	Matrice de corrélation	47
3.23	Estimation par RandomForest et consommation énergétique réelle	49
3.24	Modèle de réseau de files d'attente représentant les principales étapes du traitement d'une requête DNS.	51
3.25	Modèle approché de bind9	52
3.26	Comparaison modèle-mesures dans la plage expérimentale.	57
3.27	Extrapolation du modèle jusqu'à une charge élevée.	58

Acronymes

AFNIC Association Française pour le Nommage Internet en Coopération.

ANR Agence Nationale de la Recherche.

API Application Programming Interface.

Bind9 Berkeley Internet Name Domain.

DECODES Datacenter Energy Consumption Optimisation by DNS EvaluationS.

DNS Domain Name System.

DNSSEC Domain Name System Security Extensions.

DoH DNS Over HTTPS.

DoT DNS Over TLS.

E4C Energy4Climate.

ENPC École Nationale des Ponts et Chaussées.

ENSAE Paris École Nationale de la Statistique et de l'Administration Économique de Paris.

ENSTA Paris École Nationale Supérieure de Techniques Avancées de Paris.

FAI Fournisseur d'Accès à Internet.

IETF Internet Engineering Task Force.

IMT Institut Mines-Télécom.

INT Institut National des Télécommunications.

IP Internet Protocol.

IP Paris Institut Polytechnique de Paris.

ISC Internet Systems Consortium.

ITU-T International Telecommunication Union Telecommunication Standardization Sector.

IXP Internet Exchange Point.

LLC Last Level Cache.

PQC Post-Quantum Cryptography.

QPS Query Per Second.

RST Réseaux et Services de Télécommunications.

SAMOVAR Services répartis, Architectures, Modélisation, Validation, Administration des Réseaux.

SNMP Simple Network Management Protocol.

TLD Top Level Domain.

TLS Transport Layer Security.

TTL Time To Live.

UDP User Datagram Protocol.

Introduction

Les DNS constituent une infrastructure essentielle du fonctionnement d'Internet. Utilisés quotidiennement par des milliards de requêtes, ils assurent la traduction entre les noms de domaine et les adresses Internet Protocol (IP) nécessaires à la communication entre machines. Malgré leur rôle central, leur consommation énergétique reste encore peu caractérisée, en particulier dans le cadre de leur fonctionnement distribué.

La mesure de la consommation énergétique d'un système tel que le DNS constitue un défi méthodologique important. En effet, le processus de résolution implique plusieurs acteurs (clients, résolveurs récursifs, serveurs autoritatifs) ainsi que différents protocoles de communication. Estimer la consommation globale nécessite donc de prendre en compte l'ensemble de ces composantes. Afin de simplifier ce problème, ce travail se concentre dans un premier temps sur l'estimation de la consommation énergétique du résolveur DNS, considéré comme l'élément central du processus de résolution.

Ce stage s'inscrit dans le cadre du projet DECODES et s'appuie sur les travaux de précédents stagiaires, notamment ceux ayant étudié la consommation énergétique de Bind9 [1]. Ces études constituent une base expérimentale importante, mais présentent certaines limites, notamment un manque d'interprétabilité des modèles proposés et une difficulté à généraliser les résultats à d'autres configurations ou résolveurs.

L'objectif de ce travail est de proposer et d'évaluer différentes approches de modélisation de la consommation énergétique d'un résolveur DNS. Deux axes principaux sont étudiés. Le premier repose sur des techniques d'apprentissage automatique visant à prédire la consommation énergétique à partir de métriques système et réseau. Le second s'appuie sur un modèle analytique basé sur les réseaux de files d'attente, permettant une représentation plus structurée et interprétable du fonctionnement interne du résolveur. En parallèle, une analyse de la complexité algorithmique de Bind9 est explorée afin d'apporter un éclairage complémentaire sur les mécanismes influençant la consommation.

Les contributions de ce travail sont multiples. Elles incluent la mise en place d'une méthodologie de mesure de la consommation énergétique d'un résolveur DNS, l'étude du surcoût lié à l'instrumentation, la construction et l'évaluation de modèles de prédiction basés sur le machine learning, ainsi que le développement d'un modèle analytique fondé sur les réseaux de files d'attente. Une étude préliminaire de la complexité algorithmique de Bind9 complète également ces travaux.

Le rapport est organisé en plusieurs chapitres. Le premier présente le contexte institutionnel et l'environnement de travail au sein de Télécom SudParis. Le deuxième chapitre introduit les notions théoriques nécessaires à la compréhension du DNS et des protocoles associés. Le troisième chapitre détaille le travail réalisé, incluant la mise en place de l'environnement expérimental, les campagnes de mesures, l'analyse des résultats et les approches de modélisation développées. Enfin, une conclusion synthétise les résultats obtenus et propose des perspectives d'amélioration et de recherche.

Chapitre 1

Présentation de Télécom SudParis

1.1 Présentation de l'organisme d'accueil

1.1.1 Télécom SudParis dans son environnement international, européen et français

Télécom SudParis est une grande école d'ingénieurs française spécialisée dans les technologies du numérique, les télécommunications, les réseaux, l'informatique et la cybersécurité. Depuis 2019, elle est membre de l'Institut Polytechnique de Paris, qui rassemble plusieurs grandes écoles françaises de premier plan telles que l'École Polytechnique, Télécom Paris, l'École Nationale Supérieure de Techniques Avancées de Paris (ENSTA Paris) et l'École Nationale de la Statistique et de l'Administration Économique de Paris (ENSAE Paris). Cette intégration lui permet de bénéficier d'une forte visibilité académique et scientifique à l'échelle internationale.

Au niveau européen, Télécom SudParis participe à de nombreux projets de recherche et collabore avec des universités, laboratoires et entreprises du secteur numérique. Ses activités de recherche couvrent notamment les réseaux de communication, la cybersécurité, l'intelligence artificielle et les systèmes distribués.

Au niveau national, l'école appartient également à l'Institut Mines-Télécom (IMT), premier groupe public français d'écoles d'ingénieurs et de management dédié à la transformation numérique, industrielle et énergétique. Télécom SudParis contribue ainsi à la formation d'ingénieurs et de chercheurs capables de répondre aux enjeux technologiques actuels et futurs.

1.1.2 Télécom SudParis au sein de l'Institut Mines-Télécom et de l'Institut Polytechnique de Paris

Fondée en 1979 sous le nom d'Institut National des Télécommunications (INT), Télécom SudParis est aujourd'hui l'une des écoles de l'Institut Mines-Télécom tout en étant membre de l'IP Paris. Elle a pour mission de former des ingénieurs, chercheurs et experts du numérique grâce à un enseignement de haut niveau associé à une activité de recherche reconnue.

L'école est organisée autour de plusieurs départements d'enseignement et de recherche couvrant différents domaines du numérique, tels que les réseaux, l'informatique, les systèmes embarqués ou encore la cybersécurité. Elle entretient de nombreux partenariats avec des acteurs industriels et académiques, en France comme à l'étranger.

Durant mon stage, j'ai été accueilli sur le campus de Palaiseau, situé au sein du cluster scientifique de l'IP Paris. Bien que Télécom SudParis possède historiquement un campus

principal à Évry-Courcouronnes, une partie de ses activités d'enseignement et de recherche est aujourd'hui implantée sur le campus de Palaiseau, notamment dans les locaux partagés avec Télécom Paris.

1.1.3 Le département RST

Mon stage s'est déroulé au sein du département **RST** de Télécom SudParis. Ce département constitue le pôle d'expertise de l'école dans le domaine des réseaux informatiques et des télécommunications. Il est implanté à la fois sur les campus d'Évry-Courcouronnes et de Palaiseau.

Le département RST intervient aussi bien dans l'enseignement que dans la recherche. Il assure la coordination des formations liées aux réseaux et propose notamment des spécialisations en sécurité des systèmes et réseaux ainsi qu'en architecture et intelligence pour les réseaux.

Ses activités de recherche portent sur de nombreux domaines tels que les protocoles de communication, la cybersécurité, la qualité de service, la virtualisation des réseaux, l'optimisation des infrastructures numériques ou encore les réseaux du futur. Les enseignants-chercheurs du département sont notamment associés au laboratoire Services répartis, Architectures, Modélisation, Validation, Administration des Réseaux (SAMOVAR) et collaborent avec de nombreux partenaires académiques et industriels, à l'échelle nationale et internationale.

C'est dans cet environnement de recherche et d'innovation, spécialisé dans les réseaux et les télécommunications, que s'est déroulé mon stage.

1.2 Environnement de travail

Ce stage s'est déroulé dans un environnement de recherche académique, sensiblement différent du cadre industriel ou entrepreneurial. Réalisé au sein du département RST de Télécom SudParis, il s'inscrivait dans une démarche de recherche publique dont l'objectif principal est la production de connaissances et l'étude approfondie de problématiques scientifiques.

Contrairement à un contexte industriel, où les contraintes de temps et de rentabilité occupent souvent une place centrale, la recherche académique privilégie une approche davantage fondée sur l'analyse, l'expérimentation et la validation rigoureuse des résultats. Cette méthodologie favorise l'exploration de différentes pistes de travail et laisse une plus grande place à la réflexion scientifique.

Télécom SudParis est implantée sur deux sites principaux : Évry-Courcouronnes et Palaiseau. Mon activité s'est déroulée majoritairement sur le campus de Palaiseau, au sein des locaux partagés avec Télécom Paris. Ce campus récent offre un cadre de travail moderne et agréable, caractérisé par des espaces lumineux et des infrastructures adaptées aux activités d'enseignement et de recherche.

J'ai également été amené à me rendre ponctuellement sur le site d'Évry-Courcouronnes, où étaient hébergés les serveurs utilisés pour les expérimentations. Ces déplacements ont notamment permis la configuration initiale des équipements ainsi que des échanges avec les membres de l'équipe technique.

Enfin, je disposais d'espaces de travail sur les deux sites, partagés avec d'autres stagiaires et doctorants. Cet environnement calme et collaboratif a favorisé la concentration et le bon déroulement des activités de recherche et de développement liées au projet.

Ce cadre de travail m'a permis de découvrir le fonctionnement d'un environnement de recherche académique et de développer une plus grande autonomie dans la conduite d'un projet scientifique. La flexibilité de l'organisation, associée à un encadrement régulier, m'a offert l'opportunité d'explorer différentes approches méthodologiques tout en conservant une démarche rigoureuse. Cette expérience a constitué une introduction concrète au travail de recherche et a fourni le contexte nécessaire à la réalisation des travaux présentés dans les chapitres suivants.

Chapitre 2

Contexte théorique

Afin de comprendre les travaux réalisés au cours de ce stage, il est nécessaire de présenter les principaux concepts liés au DNS, aux protocoles utilisés pour le transport des requêtes DNS ainsi qu'au logiciel Bind9, qui constitue la plateforme d'expérimentation principale de cette étude.

2.1 Fonctionnement du DNS

Le DNS est un service essentiel d'Internet dont le rôle principal est de traduire les noms de domaine compréhensibles par les utilisateurs, tels que `www.example.com`, en adresses IP utilisables par les équipements réseau.

Le DNS repose sur une architecture distribuée et hiérarchique. Chaque niveau de cette hiérarchie est responsable d'une partie de l'espace des noms de domaine, ce qui permet au système de rester scalable malgré la taille d'Internet.

Complete DNS Lookup and Webpage Query

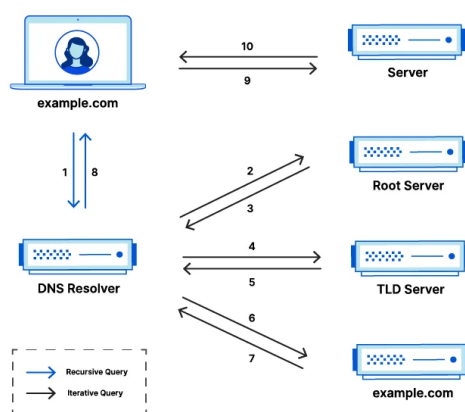


Figure 2.1 – Exemple simplifié d'une résolution DNS récursive.

2.1.1 Serveurs autoritatifs et résolveurs

L'infrastructure DNS est principalement composée de deux types de serveurs.

Les **serveurs autoritatifs** stockent les informations officielles relatives à un ou plusieurs domaines. Ils constituent la source de vérité concernant les enregistrements DNS qu'ils hébergent.

Les **résolveurs DNS** ont pour rôle de répondre aux requêtes des clients. Lorsqu'une information n'est pas disponible localement, ils interrogent successivement les différents serveurs de la hiérarchie DNS jusqu'à obtenir une réponse qu'ils retransmettent au client.

Dans le cadre de ce stage, les travaux portent principalement sur les résolveurs DNS, car ils représentent le point d'entrée des requêtes générées par les utilisateurs et intègrent des mécanismes de cache susceptibles d'influencer significativement leur consommation énergétique.

2.1.2 Résolution récursive

La résolution récursive correspond au processus par lequel un résolveur recherche une information DNS au nom du client.

Lorsqu'un utilisateur effectue une requête, celle-ci est transmise à un résolveur. Si la réponse n'est pas déjà disponible dans son cache, le résolveur contacte successivement les serveurs racine, puis les serveurs responsables du TLD concerné, avant d'interroger le serveur autoritatif du domaine recherché.

Une fois la réponse obtenue, elle est renvoyée au client et peut être conservée temporairement dans le cache du résolveur afin d'accélérer les futures requêtes similaires.

2.1.3 Le mécanisme de cache

Le cache constitue un élément fondamental du fonctionnement des résolveurs DNS. Lorsqu'une réponse est obtenue auprès d'un serveur autoritatif, elle est conservée localement pendant une durée définie par son TTL.

Si une requête identique est reçue avant l'expiration de cette durée, le résolveur peut répondre directement à partir du cache sans effectuer de nouvelles communications réseau.

Ce mécanisme permet de réduire la latence des requêtes, de diminuer la charge des serveurs autoritatifs et de limiter le trafic réseau généré par les résolutions DNS. Dans le cadre de ce stage, l'étude de l'impact énergétique du cache constitue un élément central des expérimentations réalisées.

2.2 Protocoles utilisés

Traditionnellement, les échanges DNS utilisent le protocole UDP. Cependant, l'apparition de nouvelles exigences en matière de confidentialité et de sécurité a conduit au développement de protocoles chiffrés tels que DoT et DoH.

2.2.1 DNS sur UDP

Le protocole UDP est historiquement le mode de transport le plus utilisé pour les requêtes DNS.

Son principal avantage réside dans sa simplicité : aucune connexion préalable n'est nécessaire entre le client et le serveur, ce qui réduit la surcharge protocolaire et permet d'obtenir de très faibles temps de réponse.

Cependant, les données sont transmises en clair sur le réseau, ce qui les rend observables par des intermédiaires.

2.2.2 DNS over TLS (DoT)

Le protocole DNS over Transport Layer Security (TLS) encapsule les échanges DNS dans une connexion TLS.

Cette approche garantit la confidentialité et l'intégrité des données échangées entre le client et le résolveur. En contrepartie, l'établissement et le maintien de la connexion TLS introduisent un coût supplémentaire en termes de calcul et de consommation de ressources.

2.2.3 DNS over HTTPS (DoH)

Le protocole DNS over HTTPS transporte les requêtes DNS au travers du protocole HTTPS.

En plus du chiffrement apporté par TLS, cette approche permet aux requêtes DNS de se fondre dans le trafic Web classique. Cette caractéristique améliore la confidentialité des utilisateurs mais ajoute également une couche protocolaire supplémentaire susceptible d'avoir un impact sur les performances et la consommation énergétique des serveurs.

L'une des motivations de ce stage est précisément d'évaluer l'influence de ces différents protocoles sur la consommation des infrastructures DNS.

2.3 Présentation de Bind9

Les expérimentations réalisées durant ce stage reposent principalement sur Bind9, l'un des logiciels DNS les plus utilisés au monde.

Développé par l'Internet Systems Consortium (ISC), Bind9 est une implémentation open source permettant de déployer aussi bien des serveurs DNS autoritatifs que des résolveurs récursifs. Sa large adoption dans les environnements académiques, industriels et institutionnels en fait une référence pertinente pour l'étude du comportement énergétique des infrastructures DNS.

Bind9 offre de nombreuses fonctionnalités avancées, notamment la gestion du cache, le support des protocoles DNS modernes tels que DoT et DoH, ainsi que des outils détaillés de supervision et de collecte de statistiques. Ces caractéristiques en font une plateforme particulièrement adaptée à la réalisation de mesures expérimentales et à la construction de modèles de consommation énergétique.

L'ensemble des mesures et analyses présentées dans les chapitres suivants a ainsi été réalisé à partir de différentes configurations de Bind9 exécutées sur les serveurs du projet DECODES.

Chapitre 3

Travail réalisé

Ce chapitre présente les travaux réalisés dans le cadre de mon stage de recherche au sein du projet DECODES. Ce stage s'inscrit dans un contexte scientifique portant sur l'étude des infrastructures réseau, et plus particulièrement sur l'analyse du fonctionnement du DNS ainsi que de ses différents protocoles de communication.

Les travaux effectués se situent à l'intersection de deux domaines. D'une part, le domaine des réseaux, qui comprend l'étude des mécanismes de résolution DNS, des protocoles associés (UDP, DoT, DoH) et de leur impact sur les performances et la consommation des systèmes. D'autre part, le domaine de la science des données, dans lequel mes compétences ont été mobilisées pour l'analyse des données expérimentales, la modélisation de la consommation énergétique et l'exploitation des mesures collectées.

Bien que ma formation initiale soit principalement orientée vers la data science et relativement éloignée des problématiques réseaux, les connaissances fondamentales acquises dans ce domaine ont été complétées au cours du stage afin de mieux appréhender le fonctionnement du DNS et des protocoles étudiés.

Ce chapitre a pour objectif de présenter de manière structurée les différentes missions réalisées. Il débute par une description du projet DECODES ainsi que la place que j'occupe dans ce projet et des objectifs du stage, notamment dans une perspective de poursuite en thèse. Il détaille ensuite l'organisation du travail, la méthodologie utilisée pour établir un protocole de mesure ainsi que la mise en place de l'environnement expérimental, en continuité avec les travaux réalisés par les stagiaires précédents. Enfin, il présente les principaux résultats obtenus ainsi que les analyses effectuées à partir des données collectées.

3.1 Description du projet DECODES

Le projet DECODES s'inscrit dans un contexte où la consommation énergétique des infrastructures numériques, et en particulier celle des **centres de données**, devient un enjeu majeur. Alors que les services numériques connaissent une croissance exponentielle, leur empreinte électrique devrait encore s'accroître significativement dans les années à venir. Pourtant, parmi les composants critiques d'Internet, le DNS, bien que fondamental pour le fonctionnement du réseau, reste largement sous-estimé dans les analyses de consommation énergétique. La consommation du DNS est probablement marginale par rapport à celle des centres de données, mais cela reste un exemple de consommation énergétique d'un service réseau distribué, qui est en fonctionnement continu et qui est utilisé par tous les services numériques. La méthodologie qui sera développée dans le cadre de ce projet pourra être adaptée à d'autres services distribués, tels que les systèmes de messagerie, les réseaux sociaux ou les plateformes de streaming. De plus, étant donné que tous les services nu-

mériques reposent sur le DNS, la consommation d'autres types de services pourraient être estimée à partir des données récoltées pour l'estimation de la consommation énergétique du DNS. Enfin, la consommation énergétique du DNS peut servir d'indicateur pour détecter des défaillances dans les infrastructures réseau.

3.1.1 Contexte et motivation

Le DNS, bien que représentant une fraction modeste du trafic réseau global, se distingue par son **fonctionnement continu** et son **architecture hautement distribuée**. Son rôle central dans les communications Internet, couplé à l'évolution des mécanismes de sécurité tels que le DoH, le DoT ou encore Domain Name System Security Extensions (DNSSEC), introduit des surcoûts computationnels non négligeables. Ces protocoles, bien que renforçant la confidentialité et l'intégrité des données, augmentent la charge de traitement des serveurs, et donc leur consommation énergétique.

Malgré son importance, l'impact énergétique du DNS n'a jusqu'à présent fait l'objet que de peu d'études approfondies. Cette lacune a motivé le lancement du projet DECODES, qui vise à combler ce manque en proposant une **approche scientifique rigoureuse** pour comprendre, modéliser et optimiser la consommation énergétique liée au DNS, tout en préservant ses performances et sa sécurité.

3.1.2 Objectifs

Le projet DECODES poursuit cinq objectifs principaux, présentés sur le site officiel du projet¹ :

- O1) Construire un cadre de mesure énergétique du DNS** : Développer un framework standardisé permettant de mesurer la consommation énergétique des différents composants du DNS, notamment les **résolveurs**, les **serveurs autoritaires** et les Internet Exchange Point (IXP). Cet objectif inclut le déploiement d'outils de monitoring, la définition de méthodologies de mesure reproductibles, ainsi que la garantie d'une collecte de données respectueuse de la vie privée.
- O2) Élaborer des modèles énergétiques et un environnement de test** : À partir des mesures réalisées dans le cadre de l'objectif O1, ce volet consiste à concevoir des **modèles multi-niveaux** (infrastructure, protocole, utilisateur) pour analyser finement la consommation énergétique du DNS. Ces modèles seront validés dans un **environnement contrôlé** (sandbox), tout en explorant des stratégies d'optimisation telles que le **cache**, le **roulage intelligent** ou l'ajustement des paramètres protocolaire.
- O3) Évaluer l'impact de la Post-Quantum Cryptography (PQC) dans DNSSEC** : Cet objectif vise à analyser les conséquences, en termes de performance et de consommation énergétique, de l'intégration d'**algorithmes de cryptographie post-quantique** (comme Dilithium, Falcon ou SPHINCS+) dans le protocole DNSSEC. Des outils de benchmark seront développés pour évaluer les compromis entre **sécurité**, **efficacité** et **durabilité**.
- O4) Valider les optimisations sur des infrastructures réelles** : Les stratégies d'optimisation énergétique identifiées seront testées et validées sur des **infrastructures DNS opérationnelles**, telles que celles des registres internet, des IXP ou des Fournisseur d'Accès à Internet (FAI). L'enjeu est de démontrer que les gains énergétiques peuvent être obtenus **sans altérer** la performance ou la sécurité des services.

1. Site du projet permettant de consulter les informations relatives au projet ainsi que son avancement : <https://decodes-project.fr/>

O5) Contribuer aux standards et à la science ouverte : En parallèle des travaux techniques, le projet DECODES cherche à s'impliquer activement dans les instances de standardisation, notamment auprès de l'Internet Engineering Task Force (IETF) et de l'International Telecommunication Union Telecommunication Standardization Sector (ITU-T), afin de promouvoir des **bonnes pratiques éco-responsables** pour le DNS. Par ailleurs, l'ensemble des **jeux de données, outils et modèles** produits dans le cadre du projet sera publié en **accès ouvert**, afin de favoriser la reproductibilité des résultats et leur adoption par la communauté scientifique.

3.1.3 Consortium

Le projet DECODES est un **projet collaboratif de recherche** financé par l'Agence Nationale de la Recherche (ANR) dans le cadre de l'**Appel à Projets Générique 2025** (instrument PRCE). Il rassemble quatre partenaires complémentaires :

- **Association Française pour le Nommage Internet en Coopération (AFNIC)** : représentée par **Sandoche Balakricnan**, l'AFNIC apporte son expertise en tant que registre internet des noms de domaines en .fr et acteur majeur de l'écosystème DNS en France.
- **École Nationale des Ponts et Chaussées (ENPC)** : sous la direction de **Daphné Tuncer**, l'ENPC contribue par son savoir-faire en modélisation et en analyse des systèmes distribués.
- **France-IX** : avec **Simon Muyal**, ce point d'échange Internet apporte une vision opérationnelle des enjeux liés au trafic DNS et à son acheminement.
- **Télécom SudParis** : sous la responsabilité de **Michel Marot**, cet établissement apporte son expertise en réseaux, sécurité et optimisation des protocoles.

Ce consortium pluridisciplinaire permet d'aborder le projet sous des angles **théoriques, expérimentaux et opérationnels**, garantissant ainsi une approche globale et appliquée.

3.1.4 Organisation du travail

L'organisation du travail reposait sur une large autonomie dans la conduite des activités de recherche. Les objectifs généraux étaient définis par les encadrants, mais le choix des méthodologies, des outils et des protocoles expérimentaux relevait en grande partie de mon initiative. Cette liberté m'a permis d'explorer différentes approches et d'adapter progressivement mes travaux en fonction des résultats obtenus.

Le suivi du stage était assuré au travers de réunions régulières avec mes encadrants, généralement organisées de manière hebdomadaire. Ces échanges permettaient de présenter l'avancement des travaux, de discuter des difficultés rencontrées et d'orienter les prochaines étapes de la recherche.

J'ai également participé à plusieurs réunions plénières du projet DECODES réunissant l'ensemble des partenaires. Ces rencontres ont constitué une opportunité de présenter les résultats intermédiaires obtenus, de recueillir des retours extérieurs et de mieux comprendre les enjeux globaux du projet.

Les travaux réalisés au cours du stage se sont articulés autour de quatre grandes phases :

1. Étude bibliographique

Une première phase a consisté à acquérir une compréhension approfondie du sujet à travers l'étude de la littérature scientifique relative au DNS, aux infrastructures réseau et aux méthodes de mesure de la consommation énergétique.

2. Conception de l'environnement expérimental

Cette étape a porté sur la mise en place des outils nécessaires aux expérimentations : développement de scripts de mesure, configuration des serveurs de test et validation des protocoles de collecte de données.

3. Campagnes de mesures et analyse des résultats

Les expérimentations ont ensuite été conduites de manière itérative afin d'évaluer différentes configurations et d'améliorer progressivement la qualité des mesures obtenues.

4. Modélisation et valorisation des travaux

Enfin, les résultats collectés ont été exploités afin de construire des modèles d'analyse et de documenter les conclusions du stage au travers de rapports, présentations et travaux de rédaction.

3.2 État de l'art

Dans cette partie, nous allons voir les travaux précédents sur l'étude de la consommation énergétique des résolveurs DNS, ainsi que les travaux connexes à ce qui a été étudié durant ce stage. Cette partie est importante pour comprendre les contributions de ce stage et les limites des travaux précédents. Cet état de l'art s'appuie principalement sur les travaux réalisés dans le cadre du projet DECODES. En particulier, il reprend et synthétise les analyses bibliographiques présentées par Capucine Barré et al. dans l'article [2], ainsi que celles de Nicolas Renout, Gatien Roujanski et al. dans [1]. Il intègre également les éléments de contexte et les problématiques exposés dans la présentation générale du projet DECODES. En raison des contraintes temporelles du stage, une revue bibliographique exhaustive n'a pas pu être menée indépendamment.

La consommation énergétique des infrastructures numériques est devenue un enjeu majeur avec la croissance continue des services Internet et des centres de données. Si de nombreux travaux se sont intéressés aux serveurs, au cloud computing ou aux réseaux de communication, le DNS, pourtant indispensable au fonctionnement d'Internet, reste relativement peu étudié du point de vue énergétique.

3.2.1 Méthodes de mesure de la consommation énergétique

Les premières études sur la consommation énergétique des infrastructures Internet reposaient principalement sur des modèles analytiques à grande échelle. Par exemple, Baliga et al. [3] ont proposé une modélisation hiérarchique de la consommation énergétique d'Internet en distinguant les réseaux d'accès, métropolitains et cœur de réseau. Bien que ces travaux aient permis d'identifier les principaux postes de consommation, ils reposent sur des hypothèses simplificatrices qui ne reflètent plus la complexité des infrastructures actuelles.

Plusieurs travaux ont ensuite comparé les différentes méthodes de mesure énergétique. Dudkowski et Samdanis [4] distinguent notamment les mesures matérielles directes (wattmètres), les mesures logicielles reposant sur les Application Programming Interface (API) du système d'exploitation et les approches indirectes basées sur des protocoles de supervision tels que Simple Network Management Protocol (SNMP). Les mesures matérielles offrent la meilleure précision mais sont difficiles à déployer à grande échelle, tandis que les approches logicielles sont plus facilement généralisables mais présentent des erreurs parfois importantes.

Cette limitation est confirmée par McCullough et al. [5], qui montrent que les modèles de prédiction énergétique, qu'ils soient linéaires ou non linéaires, peuvent produire des erreurs supérieures à 150% sur des architectures multicœurs modernes. D'autres études, notamment celles de Babuta et al. [6] et Jay et al. [7], concluent également que les wattmètres physiques demeurent la référence pour obtenir des mesures fiables, même si les outils logiciels permettent une analyse plus fine des différents composants matériels.

3.2.2 Consommation énergétique du DNS

Malgré l'importance du DNS dans l'écosystème Internet, peu d'études ont analysé précisément sa consommation énergétique. Parmi les travaux les plus complets, Hankel et al. [8] ont développé une méthodologie permettant de mesurer la consommation énergétique de plusieurs résolveurs DNS (Bind, Unbound et Microsoft DNS) sur différentes plateformes logicielles. Leurs résultats montrent que le processeur et la mémoire constituent les principaux contributeurs à la consommation énergétique lors du traitement des requêtes DNS. Ils mettent également en évidence l'influence significative du système d'exploitation et des paramètres de configuration sur l'efficacité énergétique du résolveur.

D'autres travaux se sont intéressés à l'impact des protocoles DNS sécurisés. Avec l'adoption croissante de mécanismes tels que DoT, DoH ou DNSSEC, le coût énergétique du DNS tend à augmenter. Le Louët et al. [9] montrent notamment que le protocole DoH peut consommer jusqu'à quinze fois plus d'énergie que le DNS classique sur UDP dans certains scénarios défavorables, principalement à cause du coût des connexions HTTPS et du chiffrement TLS. Plus généralement, plusieurs études estiment que l'utilisation de TLS peut engendrer une augmentation de la consommation énergétique de l'ordre de 30 à 35%.

Cette problématique devient particulièrement importante dans un contexte où le trafic DNS chiffré représente une part croissante des requêtes Internet. Les débats autour du chiffrement généralisé du DNS soulignent ainsi la nécessité de mieux comprendre les compromis entre sécurité, performances et consommation énergétique.

3.2.3 Positionnement du projet DECODES

Les travaux existants montrent que la consommation énergétique du DNS dépend à la fois des protocoles utilisés, des interactions entre logiciels et systèmes d'exploitation, ainsi que des caractéristiques matérielles sous-jacentes. Cependant, peu d'études analysent conjointement ces différents facteurs à l'échelle des résolveurs DNS modernes.

En revanche, peu d'études cherchent à prédire directement la consommation énergétique d'un résolveur DNS à partir des métriques internes exposées par celui-ci, telles que les statistiques de cache, l'utilisation mémoire ou les compteurs d'activité.

Le projet DECODES s'inscrit dans une démarche visant à mieux comprendre et quantifier les compromis entre performance, sécurité et consommation énergétique au sein des infrastructures DNS. Dans ce cadre, il propose la construction d'un cadre de mesure énergétique standardisé, le développement de modèles multi-niveaux (allant de l'infrastructure aux mécanismes protocolaires), ainsi que la conception d'outils d'évaluation et d'optimisation de l'efficacité énergétique. Le présent travail s'intègre dans cet ensemble en se concentrant sur l'élaboration de modèles de prédiction de la consommation énergétique des résolveurs DNS à partir de métriques collectées en temps réel. L'objectif est de fournir des estimateurs fiables et facilement déployables, complémentaires aux approches basées uniquement sur des mesures matérielles, afin de faciliter l'analyse et l'optimisation énergétique des DNS.

3.3 Dérroulement du travail

Ce stage avait pour objectif initial de mesurer efficacement et précisément un resolver DNS avec différents protocoles (UDP, DoT et DoH) avec et sans utilisation du cache en reprenant les travaux des anciens stagiaires Nicolas Renout et Gatien Roujanski. Une fois cette partie effectuée, un modèle serait établi et une partie de l'étude aurait pu porter sur la simulation de cet environnement. Il était aussi prévu de tester avec différents logiciels de résolveurs DNS et potentiellement de faire des mesures sur des résolveurs DNS réels. Cependant, la partie simulation, ainsi que les mesures sur des résolveurs DNS réels et d'autres logiciels de résolveurs DNS n'ont pas pu être réalisées durant le stage. A la place, après avoir fait les mesures de consommation énergétique du DNS avec différentes configurations de paramètres, l'étude a porté sur la conception d'un modèle de file d'attente puis sur une analyse de complexité algorithmique statique et dynamique du logiciel Bind9. Une grande partie du travail a néanmoins été consacrée à la mise en place d'une méthodologie de mesure adaptée pour faire les mesures de consommation énergétique du DNS avec et sans cache, ainsi qu'à l'analyse des résultats obtenus. Dans cette section, nous allons détailler les différentes étapes du travail réalisé, en détaillant le parcours menant à la méthodologie de mesure adoptée, la mise en place de l'environnement expérimental, les campagnes de mesures effectuées, ainsi que les analyses et modélisations réalisées à partir des données collectées.

3.3.1 Mise en place de la méthodologie de mesure

La méthodologie de mesure utilisée est inspiré par celle utilisée par les stagiaires précédents, Nicolas Renout et Gatien Roujanski. La méthodologie de mesure utilisé consiste à faire des mesures de consommations avec différents protocoles DNS (UDP, DoT et DoH), pour différents QPS (entre 60 et 600) avec et sans utilisation du cache. En reprenant la méthodologie, certains problèmes ont été identifiés, notamment le fait que les mesures de consommation énergétique du DNS avec et sans cache ne sont pas faites dans les mêmes conditions, ce qui peut fausser les résultats. Il a donc fallu trouver une nouvelle méthodologie adaptée. Cette sous-section présente la méthodologie de mesure utilisée pour faire les mesures de consommation énergétique du DNS avec et sans cache, ainsi que les modifications apportées à la méthodologie précédente.

Sélection de l'ensemble des noms de domaines utilisés

Pour faire les mesures de consommation du DNS, il faut envoyer des requêtes de résolution de noms de domaines. Un ensemble de 17 millions de noms de domaine est accessible sur GitHub avec leur popularité [10].

Avec autant de noms de domaines, on peut se demander si avoir un ensemble de 17 millions de noms de domaines est nécessaire pour faire les mesures, ou si un ensemble plus petit pourrait être suffisant. La taille de l'ensemble de noms de domaines a-t-elle une influence sur les mesures de consommation énergétique du DNS ?

Une question méthodologique importante concerne la manière de réaliser les mesures de consommation du résolveur DNS en absence de cache. Dans les travaux précédents, deux configurations extrêmes ont été utilisées : un unique nom de domaine afin de simuler un fonctionnement entièrement dépendant du cache, et un ensemble très large de noms de domaine (environ 360 000) afin de forcer l'absence d'efficacité du cache.

Cette approche permet effectivement de mettre en évidence l'impact du mécanisme de cache sur la consommation énergétique du résolveur. Cependant, elle présente certaines

limites. En particulier, elle ne correspond pas nécessairement à des scénarios d'utilisation réalistes et introduit une asymétrie entre les conditions expérimentales des deux configurations. Cette différence de contexte peut potentiellement biaiser la comparaison des résultats obtenus entre les cas avec et sans cache.

Premièrement, pour voir l'influence de la taille de l'ensemble de noms de domaines sur la consommation énergétique du DNS, il est intéressant de regarder si le cache peut être saturé, et si c'est possible à combien de noms de domaines cela correspond-t-il en moyenne ? Pour savoir cela, j'ai fait des mesures de différentes métriques associées au cache de Bind9 pour différents ensembles de noms de domaines entre 100 000 et 14 millions de noms de domaines. En premier lieu avec des ensembles de noms de domaines entre 100 000 et 1 million de noms de domaines avec un interval de 100 000 noms de domaines entre chaque mesure, puis avec des ensembles de noms de domaines entre 1 et 14 millions de noms de domaines avec un interval de 1 million de noms de domaines entre chaque mesure.

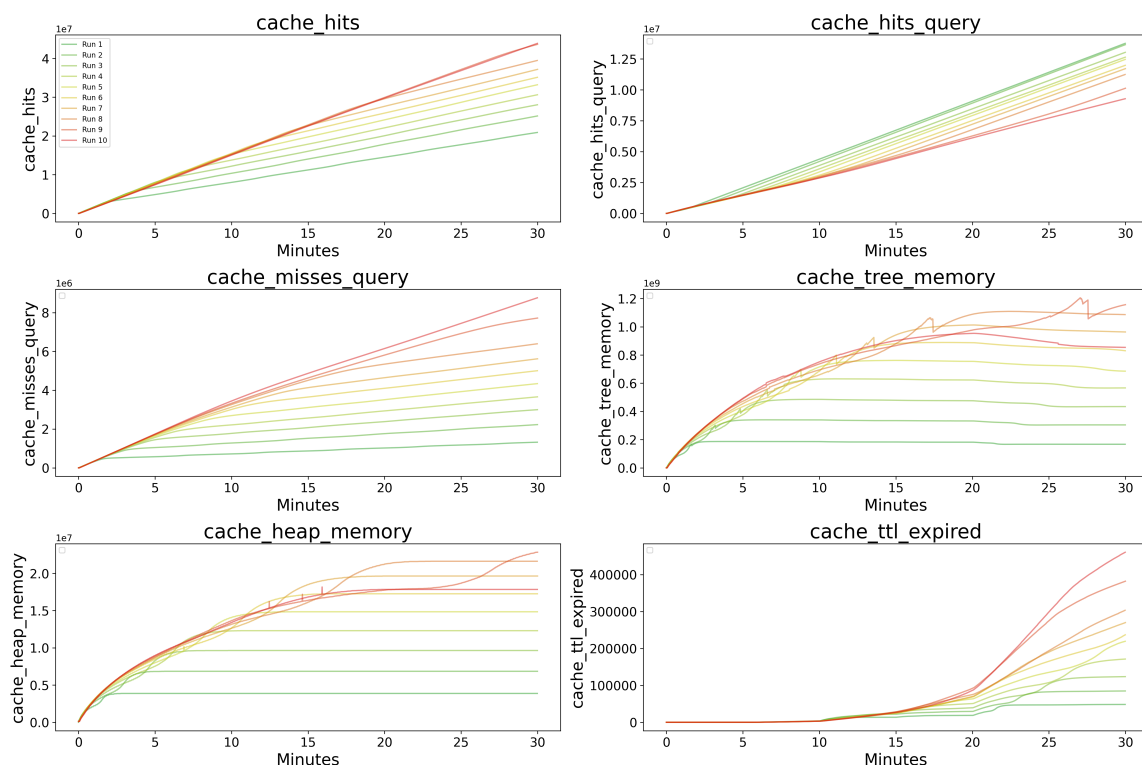


Figure 3.1 – Évolution des valeurs associées au cache de Bind9 pour différents ensemble de noms de domaines entre 100 000 et 1 millions de noms de domaines

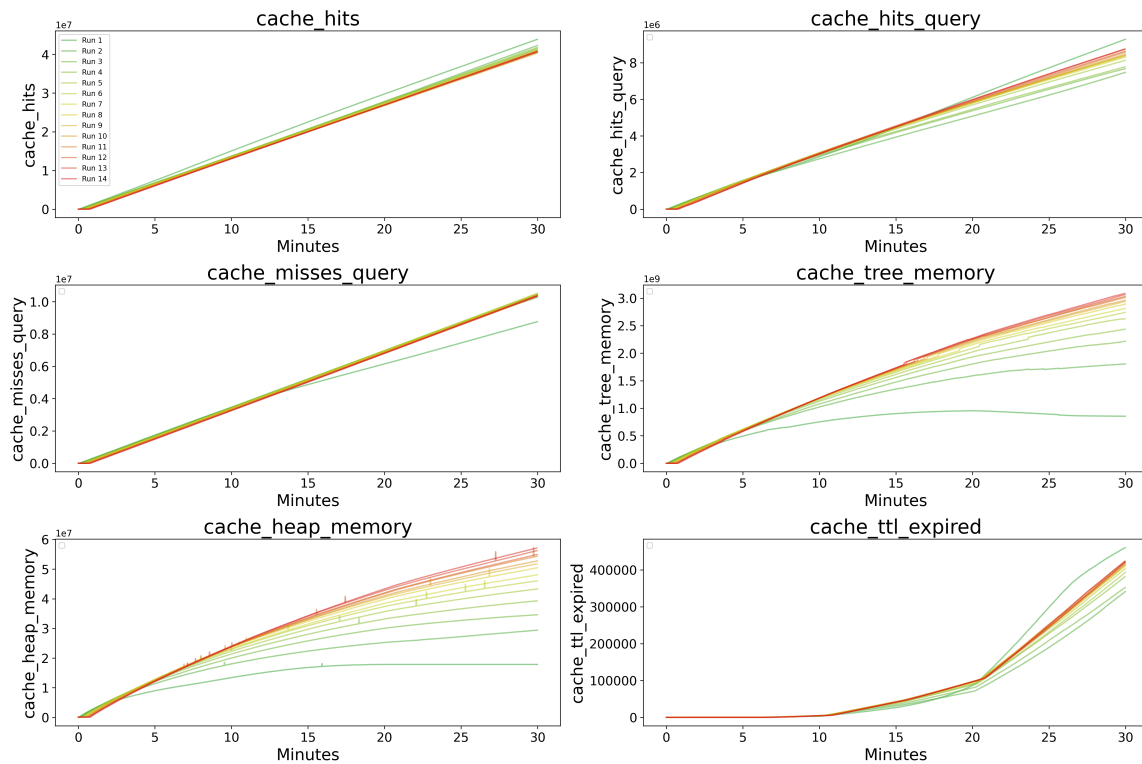


Figure 3.2 – Évolution des valeurs associées au cache de Bind9 pour différents ensemble de noms de domaines entre 1 et 14 millions de noms de domaines

Les mesures ne vont pas au-delà de 14 millions de noms de domaines car la RAM du serveur client n'était pas suffisante pour faire les mesures avec des ensembles de noms de domaines plus grands et cela ne changeait pas significativement les résultats.

Les 6 métriques mesurées sont :

- **cache_hits** : nombre total de réponses servies depuis le cache
- **cache_hits_query** : nombre de requêtes résolues directement depuis le cache
- **cache_misses_query** : nombre de requêtes nécessitant une résolution externe
- **cache_tree_memory** : mémoire utilisée par la structure arborescente du cache
- **cache_heap_memory** : mémoire utilisée par le tas du cache
- **cache_ttl_expired** : nombre d'entrées supprimées après expiration du TTL

La métrique **cache_mem_exhaust** n'est pas représentée sur les figures car sa valeur reste nulle durant l'intégralité des mesures. Ce résultat indique qu'aucune saturation du cache n'est observée, y compris pour l'ensemble de données le plus important, composé de 14 millions de noms de domaine.

L'évolution des métriques **cache_tree_memory** et **cache_heap_memory** met en évidence le remplissage progressif du cache au cours du temps. Plus la taille de l'ensemble de données est importante, plus la durée nécessaire à la mise en cache de l'ensemble des entrées est élevée.

Les expérimentations ont été réalisées pendant 30 minutes en utilisant le protocole UDP avec une charge constante de 12,000 requêtes par seconde. Une diminution progressive du nombre d'entrées présentes dans le cache est observée à partir d'environ 15 minutes, en raison de l'expiration des enregistrements associée au TTL. Ce phénomène devient particulièrement marqué après 20 minutes. Son ampleur augmente avec la taille de l'ensemble de données, ce qui s'explique par le nombre plus important d'entrées susceptibles d'atteindre leur durée de vie maximale.

Par ailleurs, le nombre de **cache_hits_query** augmente continuellement au cours des mesures pour l'ensemble des jeux de données. Cette augmentation est toutefois plus rapide

pour les ensembles de petite taille, le cache atteignant plus rapidement un état stable dans lequel une proportion importante des requêtes peut être satisfaite localement. À l'inverse, le nombre de **cache_misses_query** augmente de moins en moins vite au fil du temps.

Ces tendances apparaissent plus nettement sur les courbes de la figure 3.1 que sur celles de la figure 3.2. Étant donné que la durée des mesures est limitée à 30 minutes, les ensembles de grande taille ne permettent pas d'atteindre un niveau de remplissage du cache comparable à celui observé pour les ensembles plus réduits. Les variations des métriques sont donc plus progressives et les courbes présentent une pente plus faible durant la phase initiale de l'expérience.

Il convient de noter que, même lorsque l'ensemble des noms de domaine a été chargé dans le cache, des **cache_misses_query** continuent d'être observés. Ce comportement s'explique principalement par les requêtes supplémentaires générées lors des mécanismes de validation DNSSEC ainsi que par l'expiration régulière des entrées dont la durée de vie (TTL) est atteinte. Ces événements nécessitent la réalisation de nouvelles résolutions externes et contribuent ainsi au maintien d'un nombre non nul de défauts de cache.

L'analyse des courbes permet également d'estimer le temps nécessaire au remplissage du cache. Pour un ensemble de 100,000 noms de domaine, le cache atteint un état proche de la saturation après environ 1 minute et 30 secondes. Cette durée correspond au temps requis pour que la majorité des entrées de l'ensemble de données soient résolues puis stockées localement dans le cache du résolveur.

Pour répondre à la question de l'influence de la taille de l'ensemble de noms de domaine sur la consommation énergétique du DNS, des mesures ont été réalisées pour différents ensembles de noms de domaine entre 100,000 et 1 millions :

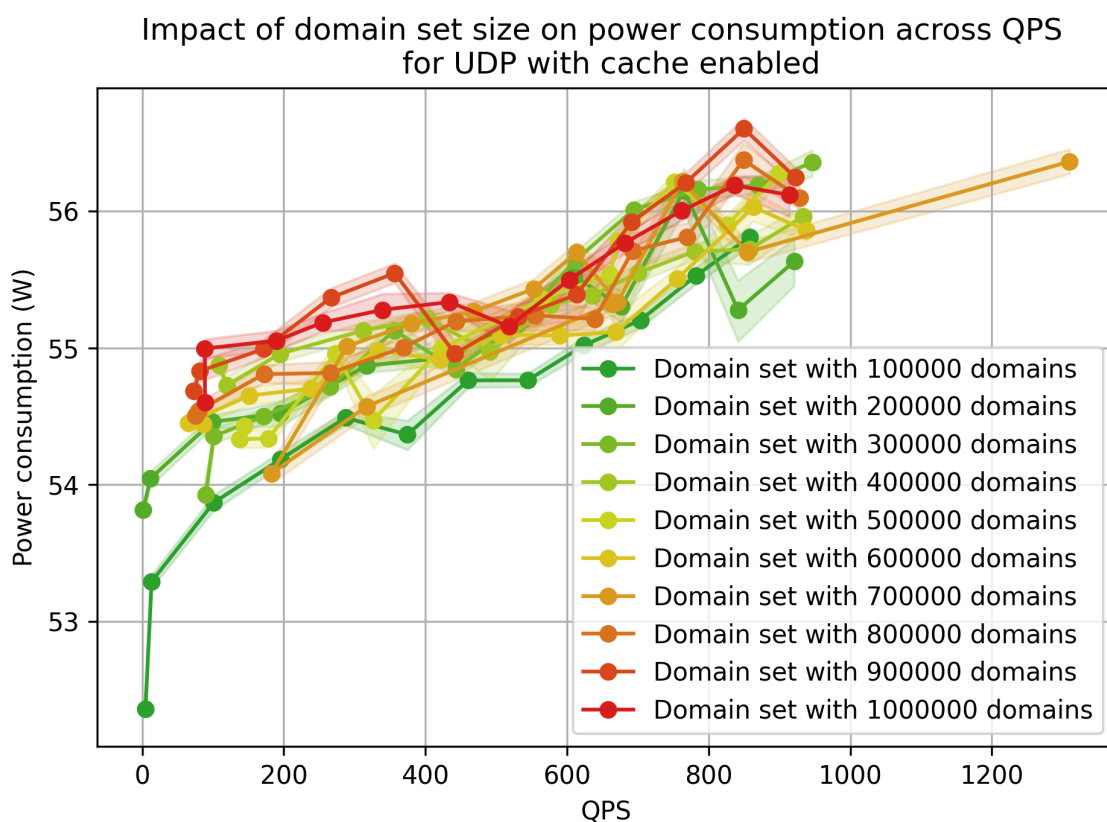


Figure 3.3 – Consommation énergétique moyenne pour différents ensembles de noms de domaine entre 100,000 et 1 million de noms de domaine

Ces mesures ont été perturbées par des IP externes envoyant des requêtes au serveur DNS durant les mesures, ce qui a faussé les résultats. Cependant, on peut observer une petite différence de consommation énergétique entre les différentes tailles d'ensemble de noms de domaine, mais cette différence n'est pas significative. Cette différence peut s'expliquer par le fait que la durée de récupération des données du cache est plus longue pour les ensembles de noms de domaine plus grands, ce qui peut entraîner une consommation énergétique légèrement plus élevée. Également, cette différence peut aussi s'expliquer par le fait que certains noms de domaines ne sont plus solvable, la proportion de ces domaines augmente avec la taille de l'ensemble de noms de domaine, cela entraîne une augmentation de la consommation car les domaines non résolus ne sont pas comptés dans le calcul du QPS.

Gestion des mesures avec et sans cache

Le protocole retenu pour la réalisation des mesures consiste, dans un premier temps, à préremplir le cache pendant une durée déterminée, suffisante pour permettre le chargement de l'ensemble des entrées associées au jeu de données de noms de domaine utilisé. Cette phase vise à garantir que les mesures effectuées avec cache reflètent le comportement du résolveur dans un état où le cache est déjà établi. Le nombre de noms de domaines choisi est 100,000 pour permettre un remplissage rapide du cache et ainsi limiter la durée totale des campagnes de mesures.

Pour les mesures réalisées sans utilisation du cache, une option de configuration de Bind9 a été employée afin de désactiver artificiellement le mécanisme de mise en cache. Il s'agit du paramètre `max-cache-ttl`, qui définit la durée de vie maximale des entrées conservées dans le cache. En fixant cette valeur à zéro, les entrées deviennent immédiatement invalides après leur insertion et ne peuvent donc pas être réutilisées pour satisfaire des requêtes ultérieures. Cette configuration permet ainsi d'émuler un fonctionnement du résolveur sans cache tout en conservant les autres mécanismes internes inchangés.

Entre chaque série de mesures, ce paramètre de configuration de Bind9 est modifié afin d'activer ou de désactiver le mécanisme de cache. Le serveur est ensuite redémarré afin de prendre en compte la nouvelle configuration et de réinitialiser l'état du cache. En complément, la commande `rndc flush` est utilisée afin de vider explicitement le cache du résolveur.

Cette commande pourrait a priori constituer une solution suffisante pour réinitialiser l'état du cache. Toutefois, les expérimentations ont montré que son exécution n'entraîne pas de modification observable de la taille du cache. Pour cette raison, le redémarrage du service Bind9 s'avère nécessaire, à la fois pour garantir la prise en compte de la configuration modifiée et pour assurer une remise à zéro effective du cache entre les différentes séries de mesures.

Mesures de la complexité algorithmique

L'orchestration des mesures de la complexité algorithmique est bien plus simple car il ne faut lancer que le script d'envoi des requêtes et le script de mesure du nombre d'appels par fonction de Bind9. Cependant un seul échantillon est réalisé pour chaque configuration, car les mesures de la complexité algorithmique sont plus stables que les mesures de consommation énergétique, et il n'est pas nécessaire de faire plusieurs échantillons pour obtenir des résultats fiables.

La mise en place d'une infrastructure d'orchestration automatisée constitue une étape essentielle pour la réalisation de campagnes de mesures à grande échelle. Elle permet de

garantir la reproductibilité des expérimentations, de limiter les erreurs humaines lors de l'exécution des protocoles et d'assurer la synchronisation des différentes sources de données collectées.

La répétition des mesures de consommation énergétique sur plusieurs échantillons permet par ailleurs de prendre en compte la variabilité inhérente aux systèmes informatiques et d'améliorer la robustesse statistique des résultats obtenus.

Enfin, la distinction entre les campagnes de mesures énergétiques et les campagnes d'analyse de la complexité algorithmique permet d'adapter les protocoles expérimentaux aux caractéristiques de chaque type de mesure. L'ensemble de cette méthodologie fournit ainsi un cadre expérimental rigoureux permettant d'obtenir des données fiables pour l'étude des performances énergétiques du résolveur DNS ainsi que pour l'élaboration des modèles présentés dans la suite de ce rapport.

3.3.2 Conception de l'environnement expérimental

Une fois la méthodologie définie, la première partie de cette recherche consiste à mesurer de manière efficace et détaillée la consommation d'un résolveur DNS. Pour ce faire, j'ai choisi une approche boîte noire, en mesurant le plus grand nombre de métriques possibles, puis en déduisant celles qui sont les plus pertinentes pour estimer la consommation énergétique.

Configuration des différentes machines

La configuration expérimentale repose sur deux machines : un client et un serveur DNS. Le client joue également le rôle d'orchestrateur et est chargé de déclencher ainsi que de coordonner l'exécution des mesures sur l'ensemble du système. La consommation énergétique externe est mesurée à l'aide d'un dispositif Yoctopuce (wattmètre) [11] et est supervisée directement depuis la machine cliente.

Le serveur DNS est basé sur une implémentation de Bind9 et exécute, en parallèle du traitement des requêtes, des scripts dédiés à la collecte de différentes métriques de performance. Ces scripts introduisent une charge supplémentaire non négligeable, susceptible d'influencer les mesures de consommation énergétique, et doivent donc être pris en compte dans l'interprétation des résultats. Dans cette architecture, le client assure ainsi simultanément les fonctions de générateur de charge et d'orchestrateur des campagnes de mesures.

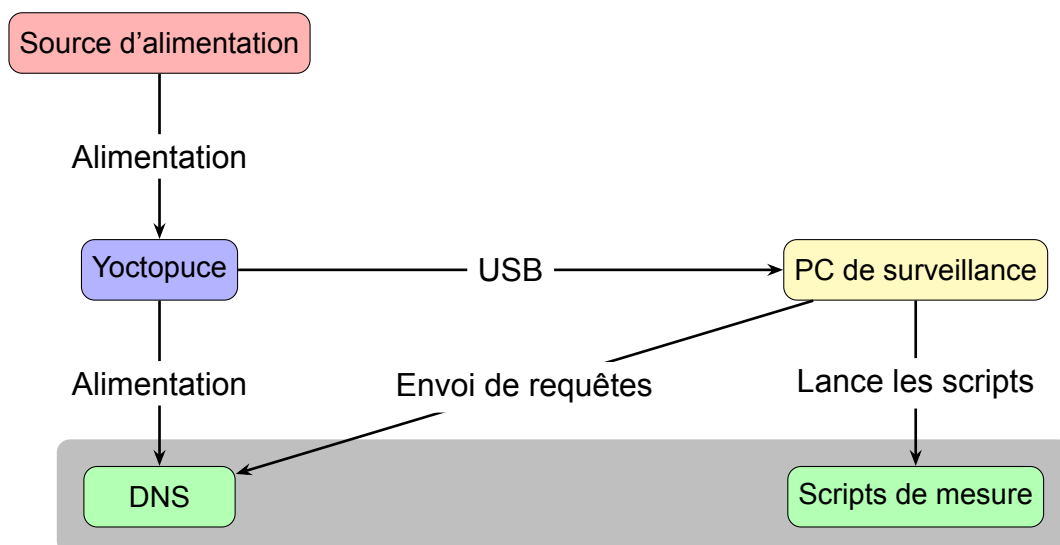


Figure 3.4 – Schéma de l'environnement expérimental

Outils de mesure utilisés

Un grand nombre de métriques sont collectées pour aider à estimer et modéliser la consommation énergétique du résolveur DNS. Ces métriques peuvent se classer dans 4 catégories : les mesures de critères systèmes, les mesures de consommation énergétique externes, les logs réseau et les mesures spécifiques au logiciel (Bind9). Pour collecter ces métriques les outils utilisés selon la catégories sont :

- **Mesures de critères systèmes** : `turbostat` (CPU), mesures système données par le kernel (I/O, NIC), `perf` (RAM)
- **Mesures de consommation énergétique externes** : `Yoctopuce`
- **Logs réseau** : `tcpdump`
- **Mesures spécifiques à Bind9** : `rndc` (Informations sur le cache de Bind9), `uftrace` (Rapports des fonctions exécutées par Bind9)

Les outils de mesures de critères systèmes ont été sélectionnés à l'aide d'un rapport de stage écrit par une étudiante ayant fait une étude comparative de différents outils de mesure de consommation énergétique de serveurs [12]. Le détail des métriques collectées par ces outils est présenté dans les annexes.

Concernant les outils de mesure spécifiques à Bind9, `rndc` est un outil de contrôle de Bind9 qui permet d'obtenir des informations sur le cache, les statistiques de requêtes, etc. `uftrace` est un outil de traçage de fonctions qui permet d'obtenir des rapports détaillés sur les fonctions exécutées par des logiciels comme Bind9, ce qui est particulièrement utile pour l'analyse de la complexité algorithmique.

Scripts de mesure développés

Afin de mesurer les métriques nécessaires à l'évaluation de la consommation énergétique du résolveur DNS et à la modélisation de celle-ci, plusieurs scripts de collecte ont été développés. Ces scripts permettent d'extraire des indicateurs système, réseau et énergétiques de manière synchronisée sur l'ensemble de la chaîne de mesure.

- **Scripts de mesure des indicateurs système** :
 - `measure_cpu.sh` : utilise l'outil `turbostat` afin de collecter des informations relatives à l'utilisation du processeur, notamment la fréquence, les états de sommeil (C-states) ainsi que les compteurs de performance associés à l'activité CPU.
 - `measure_io.sh` : exploite les fichiers `/sys/block/sda/stat`, qui contiennent des compteurs cumulés depuis le démarrage du système pour les opérations d'E/S disque (lecture et écriture en kilo-octets). Les valeurs sont différenciées temporellement afin d'obtenir les volumes d'I/O sur chaque intervalle de mesure.
 - `measure_nic.sh` : repose sur les compteurs du noyau Linux situés dans `/sys/class/net/enol/statistics/`, notamment `rx_bytes` et `tx_bytes`, représentant respectivement le volume de données reçues et transmises par l'interface réseau. Comme pour les I/O disque, une différenciation temporelle permet d'obtenir les débits sur les intervalles considérés.
 - `measure_ram.sh` : collecte des métriques liées à l'utilisation mémoire avec `perf` via les opérations sur le Last Level Cache (LLC), notamment les événements `LLC-loads` et `LLC-stores`. Ces indicateurs sont utilisés comme estimateurs indirects de l'activité mémoire et des accès à la RAM.
- **Scripts de collecte de logs réseau** :
 - `dns_activity_logger.sh` : capture le trafic réseau à l'aide de `tcpdump` afin d'analyser les requêtes DNS reçues par le résolveur ainsi que les réponses émises. Ces données permettent de reconstruire le flux de requêtes et d'évaluer la charge effective.

- `bind_query_capture.sh` : exploite les journaux de Bind9 afin d'identifier les requêtes effectivement traitées par le résolveur et d'obtenir une vision logique des résolutions effectuées (indépendamment du niveau réseau brut).
- **Script de collecte des mesures externes :**
 - `yoctowatt_log_fast.py` : utilise un dispositif Yoctopuce (wattmètre) afin de mesurer en continu la consommation énergétique du serveur exécutant le résolveur DNS. Les mesures sont synchronisées avec les autres sources de données afin de permettre une corrélation temporelle précise.
- **Mesures liées à Bind9 :**
 - `monitor_cache.sh` : interroge Bind9 via `rndc` afin d'extraire les métriques internes du cache DNS (taux de hit/miss, taille du cache, mémoire utilisée, etc.).
 - `record_ufttrace.sh` : utilise `ufttrace` pour obtenir des rapports détaillés sur les fonctions exécutées par Bind9 et leur nombre d'exécutions, ce qui permet d'analyser la complexité algorithmique dynamique et d'identifier les points chauds en termes de consommation de ressources.

L'ensemble de ces scripts constitue le squelette qui va être utilisé pour la collecte de données nécessaire à la modélisation de la consommation énergétique du résolveur DNS. Ils permettent d'obtenir une vue multi-niveaux du système (CPU, mémoire, I/O, réseau et cache DNS), ainsi qu'une mesure directe de la consommation électrique. Ces données serviront de base à la construction des modèles, à l'estimation des paramètres et à l'identification des facteurs influençant la consommation énergétique.

Détail de la configuration

Afin de garantir la reproductibilité des expériences, la version de tout les outils et protocoles utilisés durant ce stage dans le cadre des mesures sur le client et le DNS.

Catégorie	Élément	Version
OS	Linux (Debian GNU/Linux)	12 (bookworm)
Kernel	Linux kernel	6.1.0-37-amd64
DNS server	Bind9	9.18.44
DNS control	<code>rndc</code>	9.18.44
CPU metrics	<code>turbostat</code>	2025.12.02
Performance profiling	<code>perf</code>	6.1.162
Network capture	<code>tcpdump</code>	4.99.3
Function tracing	<code>ufttrace</code>	0.13

Table 3.1 – Environnement et outils utilisés sur le serveur DNS

Catégorie	Élément	Version
OS	Linux (Debian GNU/Linux)	13.3
Kernel	Linux kernel	6.1.0-21-amd64
Language	Python	3.11.2
Librairie	httpx	0.28.1
Librairie	dns	2.8.0
Librairie	yoctopuce	2.1.11632
Application	DNS	RFC 1035
Application	DoT	RFC 7858
Application	DoH	RFC 8484
Application	HTTP	HTTP/2
Transport	UDP	RFC 768
Sécurité	TLS	TLS 1.3

Table 3.2 – Environnement et outils utilisés sur le client

L'ensemble de ces éléments constitue le socle expérimental des mesures réalisées dans ce travail. La connaissance précise des outils, des systèmes d'exploitation et des protocoles utilisés est essentielle pour garantir la reproductibilité des résultats et assurer une interprétation correcte des observations effectuées sur la consommation du résolveur DNS.

3.3.3 Implémentation des campagnes de mesures

Maintenant que l'environnement expérimental est configuré et que les outils de mesure sont en place, la phase suivante consiste à réaliser les campagnes de mesures. Pour ce faire, il faut définir les différentes configurations à tester, les protocoles à utiliser, les charges à appliquer, etc. Ensuite il faut un orchestrateur pour lancer les différentes mesures de manière automatisée et synchronisée.

Il faut noter que étant donné que les campagnes de mesures sont généralement longues à réaliser, il est important de les planifier de manière efficace et de s'assurer que l'espace disque est suffisant pour stocker les données collectées. Il est également important de vérifier si la mesure est stable et fiable avant de lancer les campagnes de mesures à grande échelle, c'est ce qu'on appelle un sanity check, qui consiste à faire des mesures de test pour vérifier que tout fonctionne correctement et que les résultats obtenus sont cohérents avec les attentes. Cela est fait au préalable à chaque changement dans les scripts de mesure ou dans la configuration expérimentale. Une analyse rapide est effectuée pour vérifier le bon fonctionnement rapidement.

Le script le plus important pour toutes les mesures est le script d'envoi des requêtes. Il prend en paramètres le protocole à utiliser et la charge à appliquer. Il existe deux versions de ce script, une pour les connexions persistantes et une pour les connexions non persistantes.

Mesures de consommation énergétique

En premier lieu, chaque mesure basée sur la consommation du résolveur DNS sera échantillonnée plusieurs fois (5 fois dans notre cas) afin d'obtenir des résultats plus fiables (réduire le bruit et les incertitudes) et de pouvoir calculer des intervalles de confiance. Pour les configurations sélectionnées, on retrouve les différentes combinaisons suivantes :

- **Commun pour les deux types de connexions :**
 - Protocoles : UDP, DoT, DoH

- Utilisation du cache : avec cache, sans cache
- **Connexions persistantes** : une connexion TLS est établie au début de la mesure et est maintenue tout au long de la mesure.
 - QPS : allant de 50 QPS à 550 QPS avec 50 QPS d'intervalle entre chaque mesure
- **Connexions non persistantes** : une nouvelle connexion TLS est établie pour chaque requête DNS envoyée par le client.
 - QPS : allant de 30 QPS à 330 QPS avec 30 QPS d'intervalle entre chaque mesure

Pour l'orchestration des mesures, plusieurs scripts sont responsables de lancer les différentes mesures de manière automatisée et synchronisée. En partant du plus haut niveau au plus bas niveau, on trouve :

- `extensive_measurements.sh` : Script principal avec une boucle par paramètre de configuration (protocoles, utilisation du cache, QPS et échantillon), il lance avec ces paramètres `start.sh`.
- `start.sh` : script qui exécute une mesure unique pour une configuration donnée. Il vide le cache du DNS, change la configuration de Bind9 pour activer ou désactiver le cache, redémarre le service, crée les dossiers pour l'organisation des données récoltées, lance `start_DNS_logs_monitoring.sh` sur le DNS puis lance `start_measurements.sh` sur le client.
- `start_dns_logs_monitoring.sh` : script qui exécute tous les scripts de mesures réseau et de logs DNS en parallèle.
- `start_measurements.sh` : script qui exécute tous les scripts de mesures systèmes, réseau et énergétiques en parallèle.
- `start_measurements.sh` : script qui exécute le script de mesure de consommation externe `yoctowatt_log_fast.py` en parallèle avec `orchestrate_measures.sh` coté DNS pour lancer tout les scripts de mesure de critères systèmes.

Des campagnes de mesures ont été réalisées pour mesurer la consommation énergétique du résolveur DNS avec les mêmes configurations mais sans effectuer de mesures au niveau du résolveur DNS pour voir l'influence de ces mesures sur la consommation énergétique du résolveur DNS. Également, la consommation énergétique du résolveur DNS a été mesurée au repos pour avoir une base de comparaison pour les mesures réalisées avec les différentes configurations.

3.4 Analyse des résultats

Les mesures obtenues ont été analysées principalement avec des Jupyter notebooks en utilisant pandas, numpy et matplotlib pour visualiser les résultats.

Ces analyses ont pour but de vérifier des hypothèses émises. La visualisation est essentielle pour valider ces hypothèses et comprendre les résultats obtenus, et repérer les erreurs et anomalies dans les mesures.

Certains problèmes ont été rencontrés lors de l'analyse des résultats, notamment en raison du temps de traitement nécessaire, et la mémoire vive nécessaire pour analyser les grandes quantités de données collectées. Pour résoudre ce problème, des techniques d'échantillonnage ont été utilisées afin de réduire la taille des ensembles de données tout en conservant une représentativité suffisante pour les analyses. Cela réduit la précision des résultats, mais permet d'obtenir des analyses plus rapidement et de manière plus efficace. Un script a été développé pour automatiser ce processus d'échantillonnage et d'exportation des données dans un format plus facilement manipulable pour les analyses. Il rassemble

toutes les métriques collectées lors des mesures, rééchantillonne à 1 seconde, remplit les données manquantes en sommant les valeurs cumulatives comme le nombre de requêtes et en moyennant sur la seconde les valeurs continues, et les exporte dans un format CSV pour une utilisation ultérieure dans les notebooks d'analyse.

Grâce à ce prétraitement des données automatisé, les résultats suivant ont pu être obtenus et analysés efficacement.

Le grand nombre de métriques collectées permet de faire des analyses approfondies des différents levier de consommation. Cependant, durant ce stage, ce grand panel de métriques n'a pas été exploité à son plein potentiel et par moment, il a été difficile de trouver quelles étaient les métriques les plus pertinentes à étudier pour ce sujet de recherche.

Cette section résume les résultats les plus intéressants obtenus et les analyses, en mentionnant les limitations de ces résultats s'il y en a, et les prochaines étapes pour les améliorer ou les approfondir.

3.4.1 Mesures de consommation énergétique du résolveur DNS

Les résultats les plus concrets de cette étude de consommation énergétique du résolveur DNS sont ceux montrant la consommation énergétique du résolveur pour les différents protocoles utilisés (UDP, DoT et DoH) avec et sans utilisation du cache pour différentes charges.

Pour les protocoles DoT et DoH, les mesures ont été réalisées avec des connexions persistantes et non persistantes. Pour les connexions persistantes, une connexion TLS est établie au début de la mesure et est maintenue tout au long de la mesure. Pour les connexions non persistantes, une nouvelle connexion TLS est établie pour chaque requête DNS envoyée par le client.

Il faut noter que pour DoH, les connexions sont systématiquement persistantes, car DoH utilise le protocole HTTP qui est conçu pour maintenir des connexions persistantes. Cependant, pour les besoins de cette étude, une configuration a été mise en place pour simuler des connexions non persistantes en fermant la connexion HTTP après chaque requête DNS. Cela permet de simuler le fait que chaque requête DoH correspond à un client différent.

Connexions persistantes

Des mesures avec des connexions persistantes ont été réalisées au début de mon stage avec l'ancien protocole. Les résultats obtenus sont intéressants, notamment les mesures avec cache, ces mesures montrent le coût du DNS sans résolution récursive.

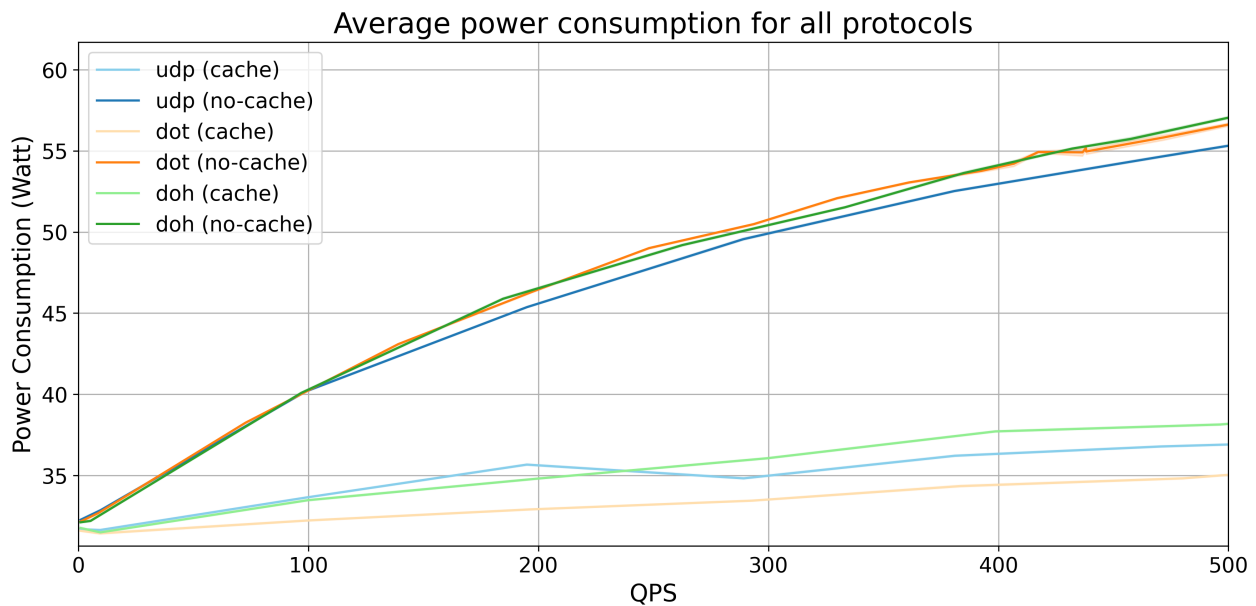


Figure 3.5 – Consommation énergétique moyenne du DNS pour UDP, DoT et DoH avec et sans utilisation du cache avec des connexions non persistantes pour l'ancien protocole de mesure montrant la consommation du DNS sans résolution récursive

Avec le nouveau protocole de mesure, la différence entre la consommation du DNS avec et sans cache est moins importante, c'est normal car l'ancien protocole de mesure avec cache correspondait au cas le moins coûteux énergétiquement.

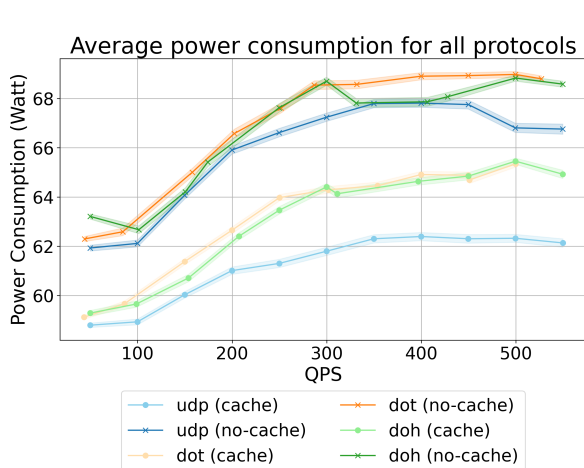


Figure 3.6 – Consommation énergétique moyenne du DNS montrant l'impact du cache et de la persistance des connexions

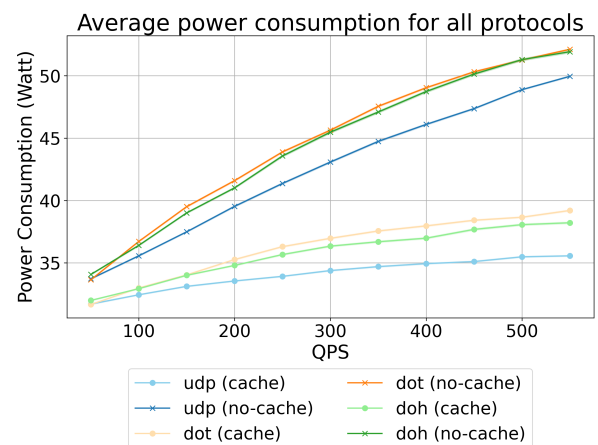


Figure 3.7 – Consommation énergétique moyenne du DNS montrant avec des connexions persistantes, sans les mesures au niveau du DNS montrant leur impact sur la consommation énergétique

Il existe une différence claire de consommation énergétique entre l'utilisation avec et sans cache 3.6. Les courbes pour l'utilisation sans cache ont une forme de décroissance exponentielle. En observant les courbes d'utilisation du cache, l'ordre est étrange : UDP n'utilise aucune connexion, il devrait donc être en dessous de toutes les autres courbes.

La différence entre les protocoles utilisant TLS (DoH et DoH) et le protocole utilisant UDP est due au coût du chiffrement principalement. Étant donné que pour ces mesures, les connexions sont persistantes, le coût de l'établissement de la connexion s'efface sur une longue période. Ces résultats sont donc cohérents avec ce qui est attendu.

Pour pouvoir interpréter ces résultats sans biais lié à la surcharge introduite par les scripts de mesure, il est nécessaire de faire des mesures de consommation énergétique sans la surcharge liée à l'enregistrement des métriques au niveau du DNS 3.7. On peut voir une grande différence dans la consommation mais également dans la forme des courbes.

Pour voir le coût de l'établissement des connexions, il faut faire des mesures avec des connexions non persistantes.

Connexions non persistantes

Avec des connexions non persistantes, la consommation énergétique est plus élevée pour les protocoles utilisant TLS (DoH et DoH) en raison du coût de l'établissement de la connexion pour chaque requête. Le protocole DoH consomme le plus ce qui est cohérent avec le coût supplémentaire du protocole HTTP en plus du chiffrement TLS.

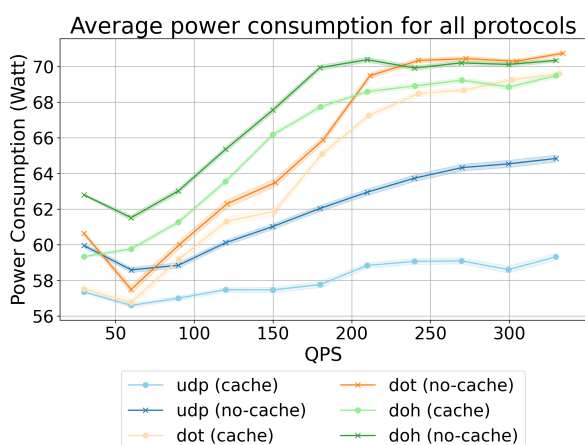


Figure 3.8 – Consommation énergétique moyenne du DNS montrant l'impact du cache et des connexions non persistantes

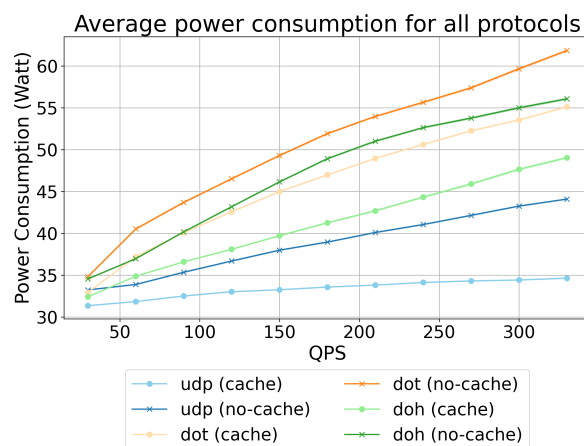


Figure 3.9 – Consommation énergétique moyenne du DNS montrant avec des connexions non persistantes, sans les mesures au niveau du DNS montrant leur impact sur la consommation énergétique

En comparant avec les résultats des mesures avec des connexions persistantes 3.8, on peut voir que la consommation énergétique est plus élevée pour les protocoles utilisant TLS (DoH et DoH) en raison du coût de l'établissement de la connexion pour chaque requête. Le protocole DoH consomme le plus ce qui est cohérent avec le coût supplémentaire du protocole HTTP en plus du chiffrement TLS.

En enlevant la surcharge liée à l'enregistrement des métriques au niveau du DNS 3.9, les résultats sont différents, le protocole DoT consomme davantage que le protocole DoH, ce qui ne corrobore pas l'hypothèse selon laquelle DoH consomme plus que DoT en raison

du coût supplémentaire du protocole HTTP. Comme le protocole mesuré était le même dans ces essais, cette différence provient probablement de la consommation des scripts de mesure au niveau du DNS, qui dépend donc du protocole utilisé.

Le QPS est limité pour DoT et DoH en raison de la surcharge introduite par l'établissement de connexions TLS pour chaque requête, ce qui rend les mesures à des QPS plus élevés difficiles à réaliser. De plus, il y a un nombre limité de sockets disponibles pour les connexions TLS, ce qui peut également limiter la capacité à atteindre des QPS plus élevés. C'est encore plus flagrant avec les connexions non persistantes, où une nouvelle connexion TLS doit être établie pour chaque requête. DoH est le protocole avec le plus de surcharge, par conséquent, c'est le protocole avec lequel le QPS maximum est le plus bas.

Il serait intéressant d'avoir des mesures pour des QPS plus élevés et avec différentes configurations DNS (nombre maximal de clients du résolveur, nombre de threads de travail, etc.) et différentes implémentations DNS (PowerDNS, Unbound, Knot DNS, etc.).

Des précédent résultats montraient moins de surcoût de la mesure de consommation énergétique. C'est à l'ajout des mesures du cache de Bind9 avec `rndc` que les résultats ont changé d'allure, c'est aussi à ce moment que la méthodologie de mesure a été revue pour bien simuler la non utilisation du cache. Il semblait plus probable que ce gros changement de comportement de consommation énergétique provienne du nouvel outil de mesure utilisé. Pour vérifier cela, une nouvelle campagne de mesure a été réalisée sans l'utilisation de `rndc`.

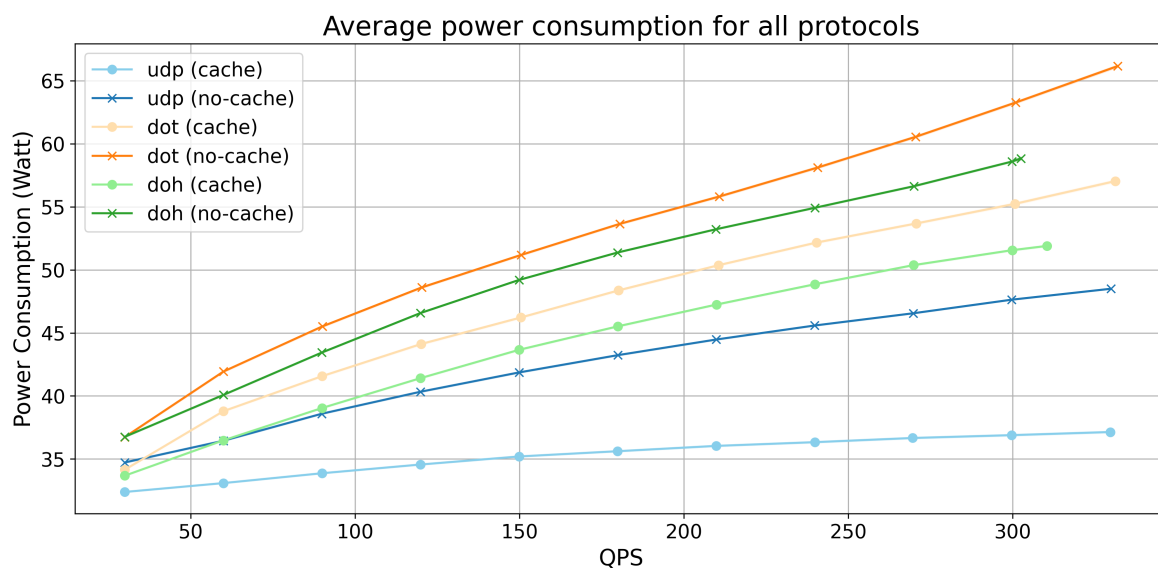


Figure 3.10 – Consommation énergétique moyenne pour UDP, DoT et DoH avec et sans utilisation du cache avec des connexions non persistantes, sans la surcharge lié à l'outil de mesure `rndc`

Ces résultats 3.10 permettent de confirmer que l'outil de mesure `rndc` change drastiquement la forme de la courbe de consommation énergétique du résolveur DNS. Cela pose la question pour les autres outils de mesure utilisés. L'impact ne serait forcément aussi significatif mais il est important de mesurer l'impact des outils de mesure sur la consommation énergétique du système étudié.

3.4.2 Mesures de métriques liées au CPU

De nombreuses métriques collectées sont liées au CPU grâce à l'outil `turbostat`. Ces métriques sont intéressantes car c'est le CPU qui consomme le plus d'énergie. Visualiser certaines de ces métriques permet de mieux comprendre le comportement de la consommation énergétique du résolveur DNS. Les métriques les plus intéressantes sont la fréquence moyenne du CPU, le pourcentage de temps d'activité du CPU, le pourcentage de temps passé dans l'état C6 (état de sommeil profond) et la température du CPU. Ces métriques sont représentées sur les figures suivantes pour les différents protocoles avec et sans utilisation du cache.

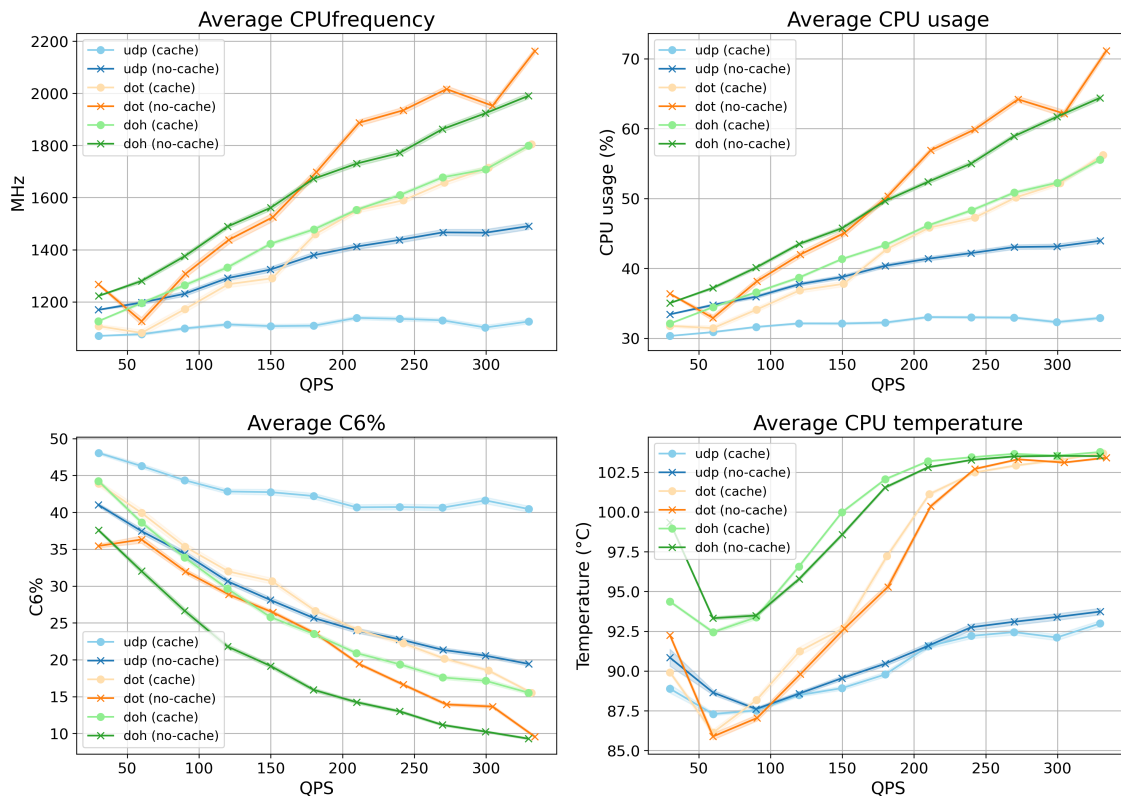


Figure 3.11 – Métriques du CPU les plus pertinentes montrant leur corrélation avec la consommation énergétique du résolveur DNS

On peut voir que la fréquence moyenne du CPU et le pourcentage de temps d'activité du CPU suivent les memes courbes, cela est du au Dynamic Voltage and Frequency Scaling (DVFS) qui ajuste dynamiquement la fréquence et la tension du CPU en fonction de la charge de travail. Pour ce qui est de la mesure de l'état C6, on peut voir que le pourcentage de temps passé dans l'état C6 est inversement proportionnel à la charge de travail du CPU. Enfin, la température du CPU suit de très proche les mesures de consommation énergétique du résolveur DNS de plus, on remarque que les températures sont extrêmement élevées avec au maximum 103°C et 86°C au minimum, ce qui est très élevé pour un CPU.

3.4.3 Mesures de métriques liées au cache de Bind9

Pour vérifier que la méthodologie utilisé pour les mesures représente bien ce que nous essayons de mesurer, il faut vérifier certaines métriques importantes. Le paramètre le plus crucial des configurations de mesure est l'utilisation ou non du cache. En effet, si le cache est

utilisé, la consommation énergétique du résolveur est beaucoup plus faible, et les résultats sont très différents de ceux obtenus sans utilisation du cache. Par conséquent, il est essentiel de vérifier que le cache est bien utilisé ou non utilisé selon la configuration de mesure. Une analyse du cache a donc été faite grâce aux métriques du cache de Bind9 collectées avec `rndc`, parmi ces métriques figurent la place occupée par le cache, pour cette mesure il existe 2 métriques importante, la mémoire utilisée par le tas du cache, c'est à dire les le contenu des données DNS et la mémoire utilisée par la structure arborescente du cache. On retrouve également le nombre de requêtes qui touchent le cache, c'est à dire le nombre de requêtes pour lesquelles une réponse est trouvée dans le cache, et le nombre de requêtes qui ne touchent pas le cache. On peut déduire avec ces dernières le pourcentage de requêtes qui touchent le cache. Enfin une métrique importante qui révèle l'efficacité de la méthodologie de mesure mise en place pour bien simuler la non utilisation du cache est l'expiration des entrées du cache due auTTL. En effet, comme mentionné précédemment, pour simuler la non utilisation du cache, leTTL maximum du cache est défini à 0, ce qui signifie que les entrées du cache expirent immédiatement après leur ajout. Par conséquent, si le cache est correctement configuré pour ne pas être utilisé, il devrait y avoir une grande quantité d'expiration d'entrées du cache due auTTL.

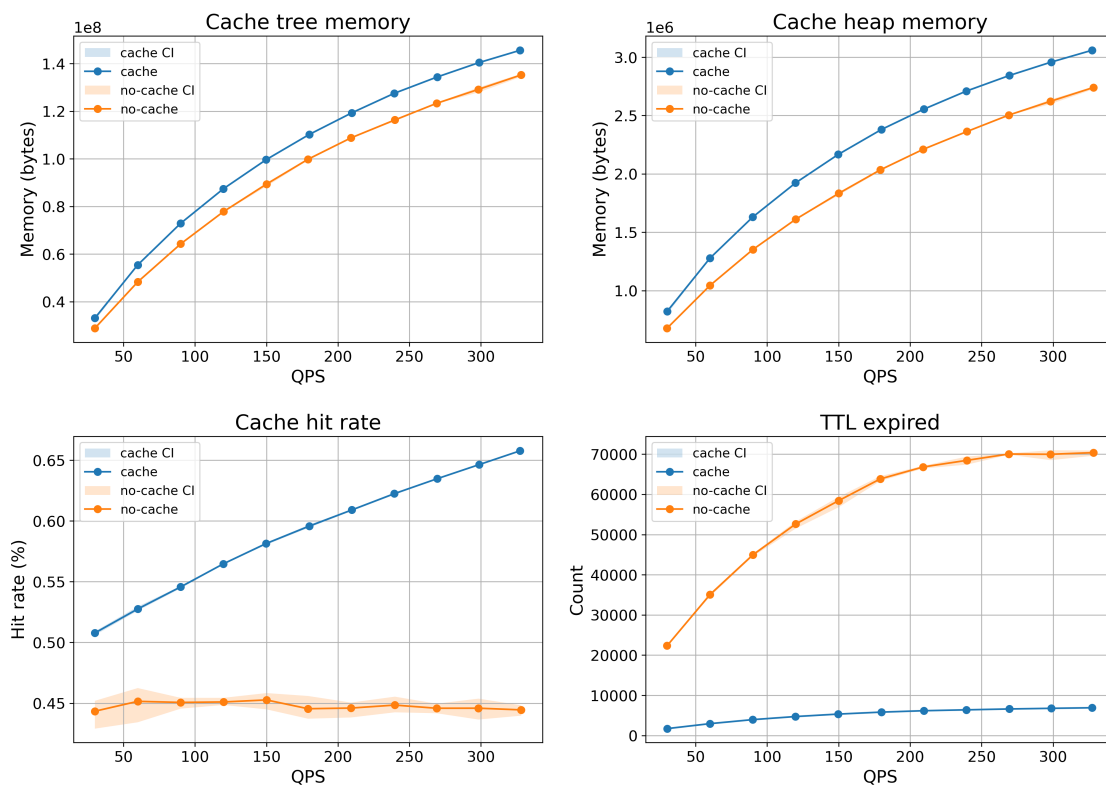


Figure 3.12 – Métriques du cache de Bind9 moyennées pour les différents protocoles avec et sans utilisation du cache

Concernant l'espace mémoire occupé par l'arborescence et le tas du cache, on peut voir une petite différence entre les mesures avec et sans utilisation du cache, mais cette différence n'est pas très grande, ce qui est surprenant. De plus, l'occupation de la mémoire augmente avec le QPS dans les deux cas, ce qui n'était pas attendu également. Cependant, on voit, comme prévu au niveau du pourcentage de requêtes qui touchent le cache une différence claire entre l'utilisation du cache et la non utilisation du cache. De plus, pour les mesures n'utilisant pas de cache on voit une stagnation du pourcentage de requêtes qui

touchent le cache vers 45% ce qui est plus que prévu. Les courbes d'expiration des entrées du cache dues au TTL montrent également une différence claire entre les mesures avec et sans utilisation du cache, on remarque une grande quantité d'expiration pour les mesures sans utilisation du cache et très peu pour l'utilisation du cache ce qui montre l'efficacité du paramètre `max-cache-ttl`.

Il faut noter que ces mesures ont été réalisées avec une erreur dans le protocole de mesure. L'étape consistant à remplir le cache avant de commencer les mesures n'a pas été faite correctement. Normalement, le cache devrait être rempli de telle sorte à ce que tous les noms de domaines soient présents dans le cache avant de commencer les mesures. Cela est censé être fait en envoyant pendant un certain temps avec un QPS fixé, dans les mesures effectuées, cette durée correspond à 1 minute 30 secondes avec 10 000 QPS pour un 100 000 noms de domaines 3.1. Cependant, lors de ces mesures, le QPS utilisé pour remplir le cache était le même que celui utilisé pour les mesures, ce qui signifie que le cache n'était pas rempli de manière homogène. Cette erreur explique l'évolution des courbes d'occupation de la mémoire du cache et du cache hit. Ces courbes sont censé rester constantes.

Maintenant que cela est dit, même avec cette erreur, on peut toujours analyser certains résultats.

Concernant la mémoire utilisée par le cache lors des mesures faites sans cache, les entrées du cache sont ajoutées au cache mais expirent immédiatement après leur ajout, elles sont supprimées du cache par un autre processus qui vérifie régulièrement quelles entrées ont expiré. Ce processus de suppression des entrées expirées du cache n'est pas instantané et il est possible que ce soit pour cette raison que l'espace mémoire occupé par le cache soit du même ordre de grandeur avec et sans utilisation du cache. Pour expliquer le pourcentage de requêtes qui touchent le cache sans utilisation du cache, il faut noter que les requêtes qui touchent le cache sont celles pour lesquelles une réponse est trouvée dans le cache ou dans la zone (serveur root uniquement dans notre cas). Cette valeur est élevée mais n'augmente pas à la différence de l'utilisation du cache, ce qui montre un comportement différent. Cette valeur peut s'expliquer par le fait que l'adresse du serveur root est nécessaire pour toutes les résolutions.

3.4.4 Étude des requêtes envoyées et reçues par le résolveur DNS

Une manière de visualiser l'utilisation du résolveur consiste à examiner chaque paquet envoyé et reçu par le résolveur. Sachant que Bind9 utilise UDP pour communiquer avec les DNS autoritatifs, l'examen des paquets UDP est idéal. Ces paquets incluent de nombreux types de requêtes que nous verrons juste en dessous. Ces nombres de requêtes sont divisés par le QPS résolu par le résolveur.

Pour les mesures utilisant UDP, comme le résolveur utilise également UDP, pour éviter de mesurer les requêtes envoyées et reçues par le résolveur, nous déduisons 2 requêtes pour chaque résolution, ce qui correspond au modèle UDP.

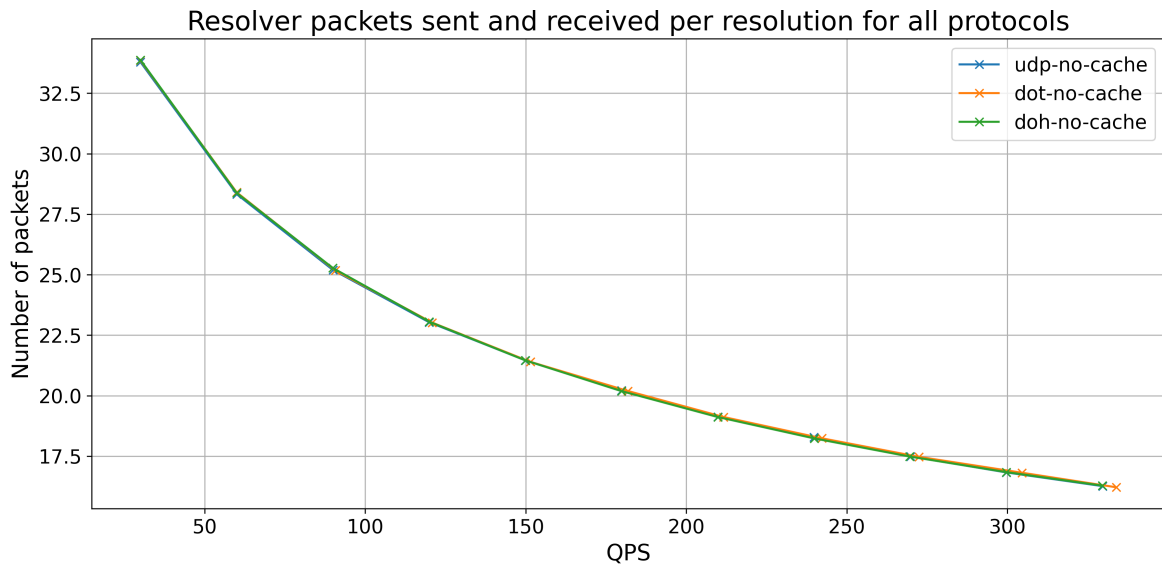


Figure 3.13 – Nombre de requêtes envoyées et reçues par le résolveur pour une résolution montrant une décroissance exponentielle avec le QPS

Cette forme de courbe suggère que plus le QPS est élevé, moins le résolveur doit envoyer (et recevoir) de requêtes. Cela s'explique par le processus de batching qui permet de traiter plusieurs requêtes pour le même nom de domaine simultanément.

Toutes les requêtes récursives ne concernent pas des adresses IP : certaines sont des délégations vers un autre serveur autoritatif (NS), d'autres sont des alias vers un autre nom de domaine (CNAME). Il y a également des informations DNSSEC telles que RRSIG, DNSKEY, DS, etc. Pour illustrer ces différents types de requêtes, voici un graphique montrant la distribution des types de requêtes envoyées et reçues par le résolveur.

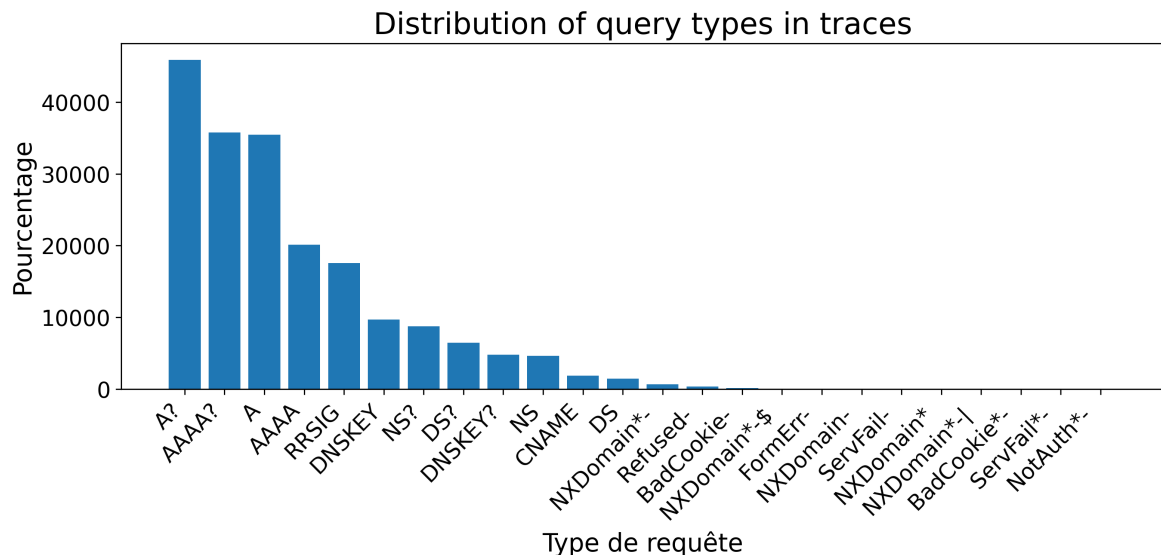


Figure 3.14 – Distribution des types de requêtes pour les résolutions récursives

La plupart sont de type A, mais il y a une part non négligeable de DNSSEC, NS et CNAME. Les types sont expliqués dans les annexe 3.5.5.

Le cache permet d'économiser beaucoup de consommation énergétique, mais les entrées du cache ont une durée limitée spécifiée par les serveurs DNS autoritatifs². Il s'agit

2. Cela est dû au fait que l'adresse IP peut changer au fil du temps. De plus, les DNS sont généralement

du TTL, spécifié en secondes dans la zone pour chaque entrée sur laquelle le DNS a autorité. La distribution de ce TTL est intéressante à examiner.

Tout d'abord, nous pouvons examiner des noms de domaine aléatoires. Le graphique de gauche montre la fonction de répartition de 100 000 noms de domaine aléatoires issus d'un ensemble d'1 million de noms de domaine pour avoir une idée de cette distribution. Le graphique de droite montre la distribution des TTL pour les TLD (comme .fr, .com, ...), au total 1577.

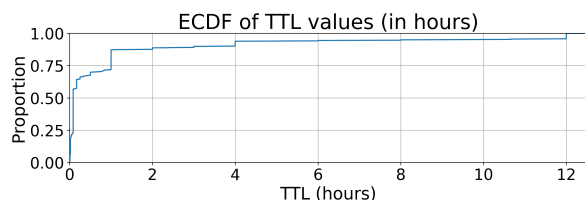


Figure 3.15 – Distribution des TTL pour des noms de domaine aléatoires

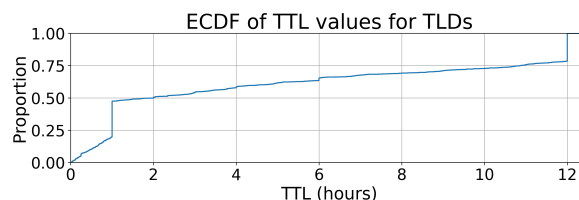


Figure 3.16 – Distribution des TTL pour les TLD

La distribution des noms de domaines aléatoire est inégale 3.15, on retrouve la majorité des noms de domaines en dessous de 1 heure, le reste des TTL ne dépassent pas 12 heures.

Pour les TLD 3.16, des pics peuvent être observés à 1h et 12h. Avec 25% des TTL de moins d'une heure, 25% de une heure, 25% entre 1h et 12h et 25% de 12h. On remarque tout de même que les TTL pour les TLD sont en moyenne plus élevés que pour les autres noms de domaine.

3.4.5 Étude de la complexité algorithmique de Bind9

Les mesures de consommation énergétique du résolveur DNS sont très intéressantes, mais elles ne permettent pas de comprendre les raisons de cette consommation énergétique. Pour mieux comprendre les raisons de cette consommation énergétique, et en s'abstrayant du matériel, une solution consiste à étudier la complexité algorithmique de Bind9. En effet, la consommation énergétique d'un programme est liée à sa complexité algorithmique, des paramètres de consommations peuvent être déduits à partir des performances énergétiques des composants de la machine (CPU, RAM, etc.).

L'étude de la complexité algorithmique de Bind9 s'est faite en deux parties, d'abord une étude de la complexité algorithmique statique de chaque fonction de Bind9 grâce à un outil développé en python par Rosario Patanè, un de mes collègues et membre du projet DE-CODES. La seconde partie de l'étude de la complexité algorithmique de Bind9 s'est faite de manière dynamique, en mesurant le nombre d'appels par fonction de Bind9 pour différentes configurations de mesure. Cela a permis d'avoir une idée de la complexité algorithmique réelle de Bind9 pour différentes configurations, et ainsi d'avoir une meilleure compréhension des raisons de la consommation énergétique du résolveur DNS.

utilisés comme une première couche de répartiteurs de charge

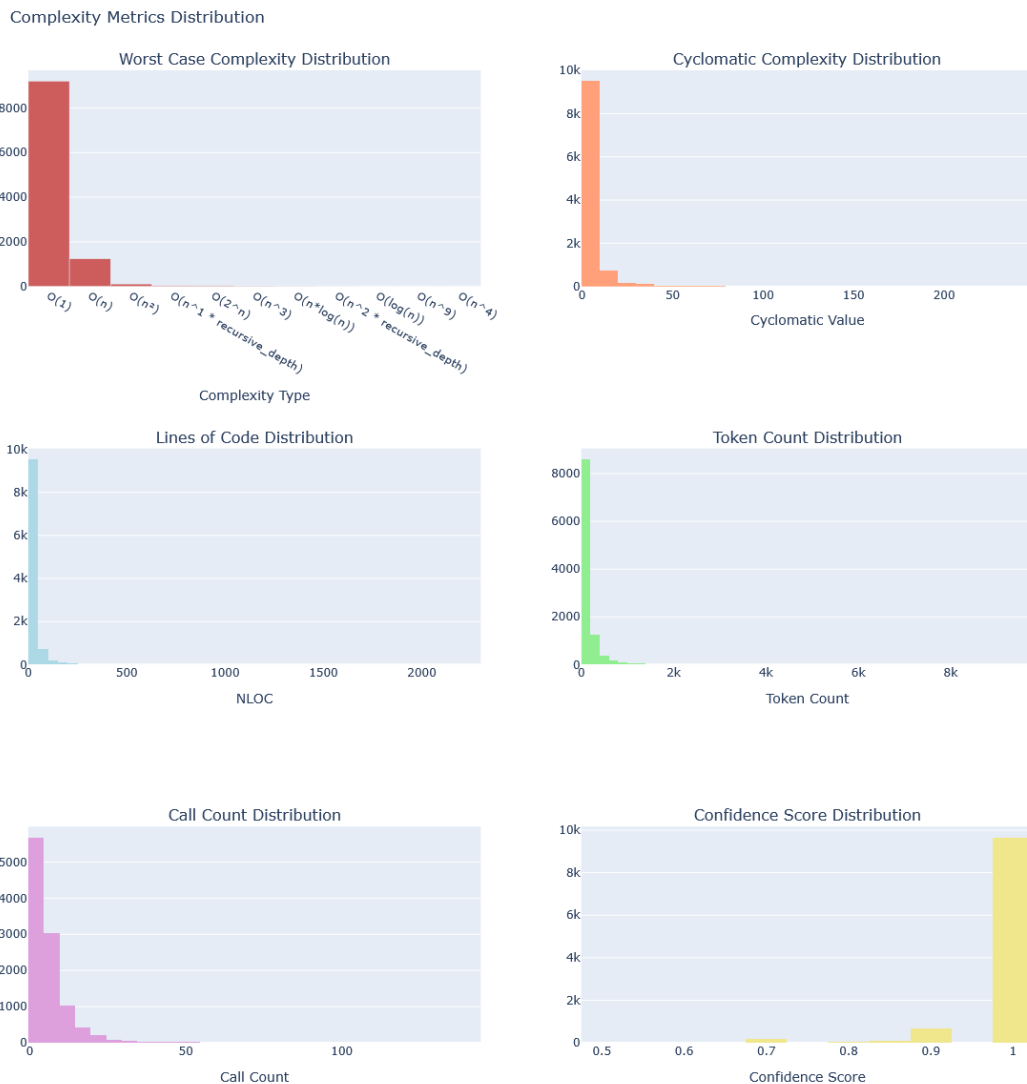


Figure 3.17 – Graphiques de l'analyse de la complexité algorithmique statique de Bind9

La figure 3.17 présente les résultats de l'analyse statique de la complexité algorithmique de Bind9 à travers six distributions complémentaires.

La première distribution représente les scénarios de pire cas (*Worst Case Scenario*) estimés pour les fonctions du code source [Bind9Code]. Elle met en évidence que la majorité des fonctions présentent une complexité faible, tandis qu'un nombre limité de fonctions se distingue par une complexité potentiellement élevée. Cette répartition est caractéristique des grands projets logiciels, dans lesquels une faible proportion de fonctions concentre une part importante de la complexité algorithmique.

La deuxième distribution concerne la complexité cyclomatique. Cette métrique mesure le nombre de chemins d'exécution indépendants à travers une fonction et constitue un indicateur de la complexité structurelle du code. Une valeur élevée traduit généralement la présence de nombreux branchements conditionnels, boucles ou mécanismes de gestion d'erreurs, rendant la fonction plus difficile à maintenir et à tester. Les résultats montrent que la majorité des fonctions de Bind9 possèdent une complexité cyclomatique relativement faible, tandis qu'un nombre restreint de fonctions présente des valeurs significativement plus élevées.

Les distributions du nombre de lignes de code et du nombre de jetons (*tokens*) permettent d'évaluer la taille des fonctions selon deux perspectives complémentaires. Alors que les

lignes de code donnent une estimation de la longueur des fonctions, le nombre de jetons fournit une mesure plus précise de leur richesse syntaxique. Dans les deux cas, les distributions mettent en évidence une forte concentration de petites fonctions et un nombre limité de fonctions particulièrement volumineuses.

La distribution du nombre d'appels entrants (*Call Count*) indique combien de fonctions différentes peuvent invoquer une fonction donnée. Cette métrique permet d'identifier les fonctions centrales dans l'architecture logicielle. Les résultats montrent que quelques fonctions jouent un rôle structurant au sein de Bind9 et sont réutilisées par de nombreuses autres fonctions.

Enfin, la distribution du score de confiance reflète le niveau de fiabilité attribué aux estimations de complexité produites par l'outil d'analyse statique. Les valeurs observées permettent d'évaluer la robustesse des résultats et d'identifier les estimations nécessitant une interprétation plus prudente.

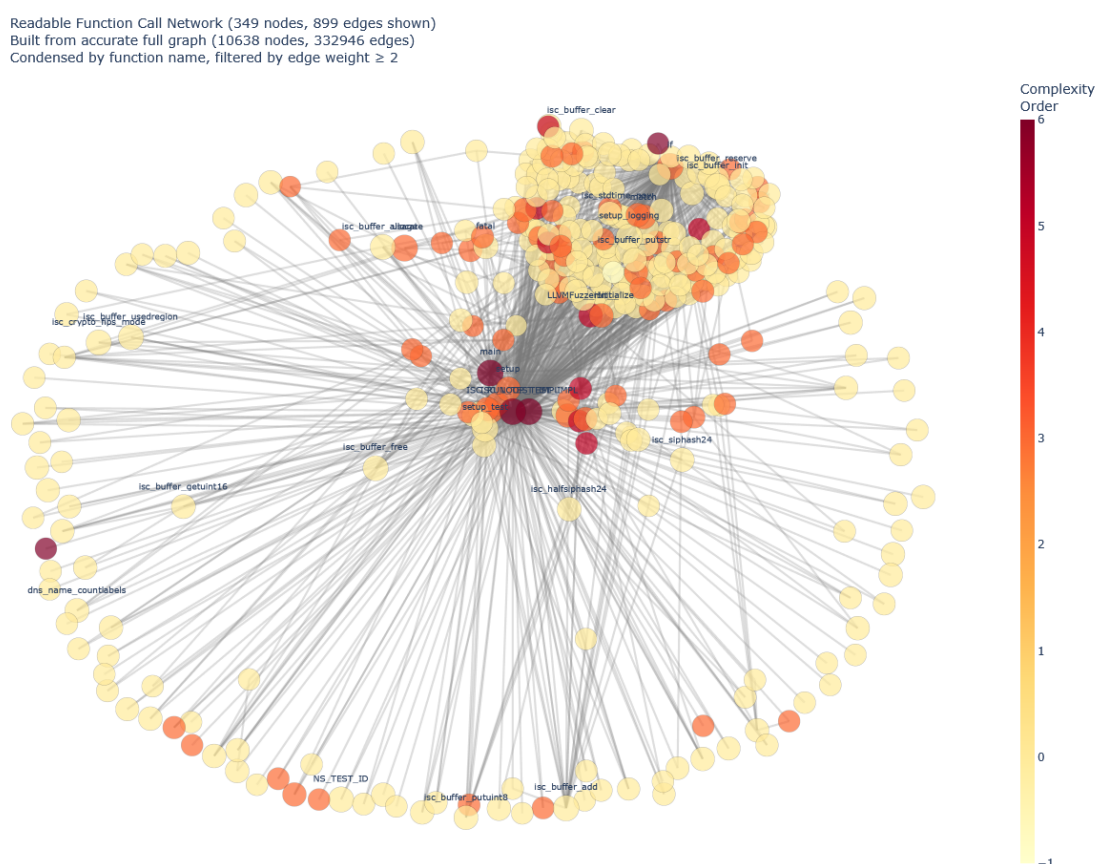


Figure 3.18 – Graphe des relations d'appels entre les fonctions de Bind9 avec leur complexités algorithmiques statiques

La figure 3.18 représente le graphe des relations d'appels entre les fonctions de Bind9, construit à partir de l'analyse statique du code source et enrichi par les métriques de complexité algorithmique associées à chaque fonction. Chaque nœud correspond à une fonction et chaque arête représente une relation d'appel identifiée dans le code. Cette représentation permet de visualiser simultanément la structure des dépendances entre fonctions et la répartition de la complexité au sein du logiciel. Les fonctions fortement connectées apparaissent comme des éléments centraux de l'architecture, tandis que les fonctions présentant une complexité algorithmique élevée peuvent être identifiées comme des zones potentiellement plus difficiles à maintenir, tester ou analyser. Ce graphe fournit ainsi une vue d'ensemble de

l'organisation statique de Bind9 et de la manière dont la complexité est distribuée dans son code source.

La suite des graphiques se concentre sur l'analyse dynamique de la complexité algorithmique de Bind9.

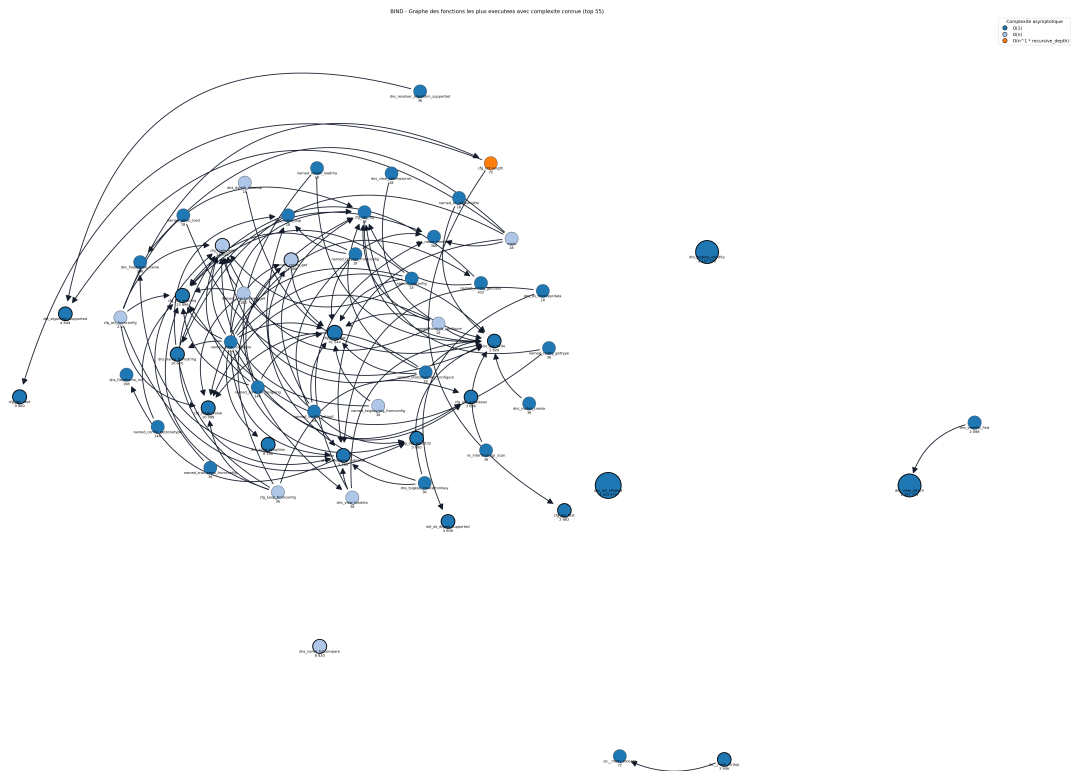


Figure 3.19 – Graph d'appels des 55 fonctions les plus appelées de Bind9 avec leur complexité algorithmique statique

La figure 3.19 se concentre sur les 55 fonctions recevant le plus grand nombre d'appels au sein de Bind9. Cette vue simplifiée facilite l'identification des fonctions critiques du système en supprimant les fonctions périphériques moins sollicitées. L'analyse met en évidence un nombre réduit de fonctions particulièrement centrales qui constituent les principaux points de passage lors de l'exécution du résolveur. Ces fonctions représentent des candidates privilégiées pour les analyses de performance et de consommation énergétique.

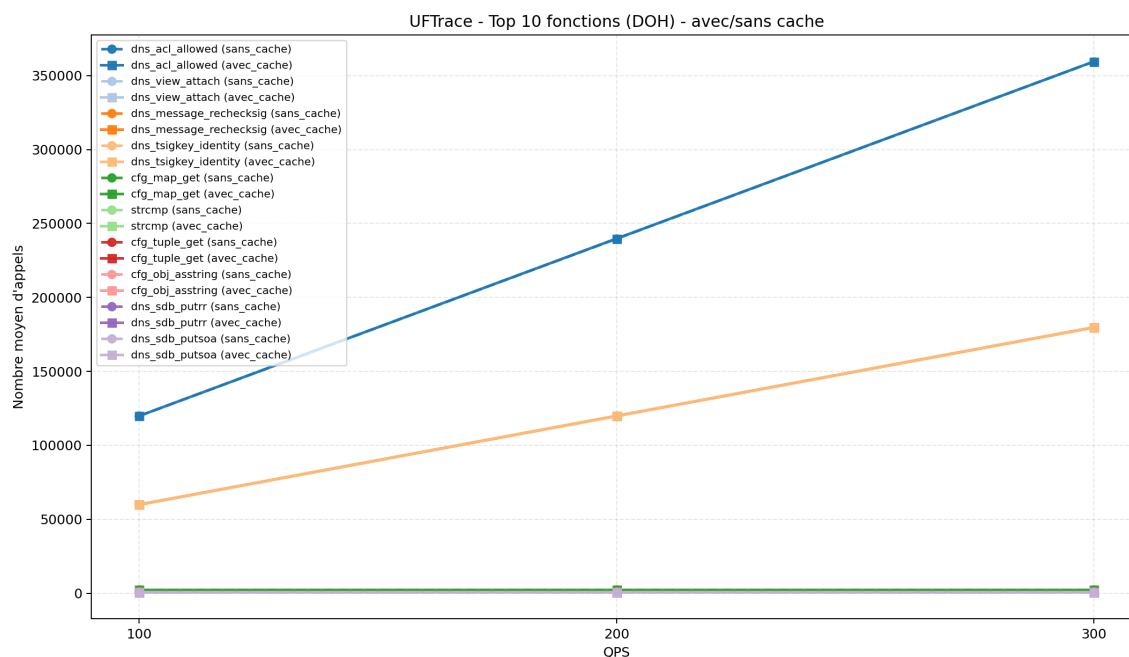


Figure 3.20 – Nombre d'appels des 10 fonctions les plus appelées de Bind9 en fonction du QPS pour DoH avec utilisation du cache

La figure 3.20 présente l'évolution du nombre d'appels des dix fonctions les plus sollicitées de Bind9 en fonction du débit de requêtes (QPS) dans le cas du protocole DoH avec utilisation du cache. Une augmentation globalement linéaire du nombre d'appels est attendue lorsque la charge augmente, traduisant la proportionnalité entre le trafic traité et l'activité interne du résolveur. Les différences observées entre les fonctions reflètent leur rôle respectif dans la pile de traitement DoH. Il n'y a aucune différence entre les mesures faites avec et sans cache, la différence doit se voir dans les temps d'exécution de ces fonctions cependant.

QPS	dns_message_rechecksig	dns_view_attach	dns_acl_allowed
100	59 943	59 945	119 886
200	119 842	119 844	239 684
300	179 724	179 726	359 448

Table 3.3 – Nombre d'appels des fonctions les plus appelées de Bind9 en fonction du débit de requêtes (QPS) pour une durée de mesure de 10 minutes.

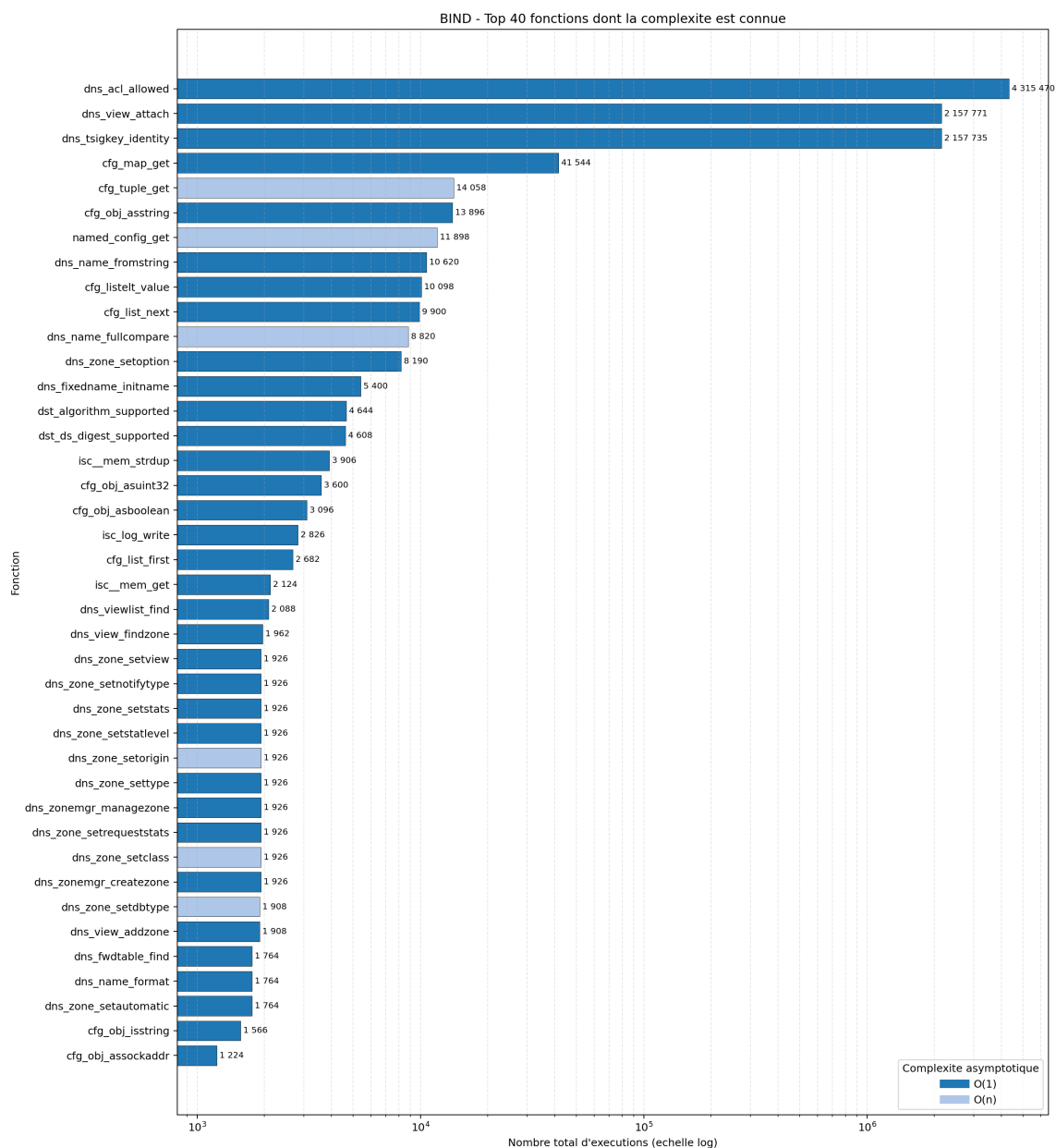


Figure 3.21 – Nombre d'appels des 40 fonctions les plus appelées de Bind9 avec leur complexité algorithmique statique

On remarque que 3 fonctions sont particulièrement sollicitées, à savoir `dns_acl_allowed`, `dns_view_attach` et `dns_tsigkey_identity`. Ces fonctions représentent à elle seule plus de 97% des appels effectués par Bind9. Elles semblent être appelées au moins une fois par requête. Voici une brève description de ces fonctions ainsi que de la fonction `cfg_tuple_get` qui est également très sollicitée, mais dans une moindre mesure.

- `dns_acl_allowed` (**$O(1)$**) : vérifie si une requête ou un client est autorisé selon les règles de contrôle d'accès (ACL) configurées dans Bind9.
- `dns_view_attach` (**$O(1)$**) : associe une nouvelle référence à une vue DNS (**view**) afin de permettre son partage sécurisé entre plusieurs composants du serveur.
- `dns_tsigkey_identity` (**$O(1)$**) : retourne l'identité (nom DNS) associée à une clé TSIG utilisée pour l'authentification des échanges DNS.
- `cfg_tuple_get` (**$O(n)$**) : recherche et récupère un élément dans une structure de configuration de type **tuple**. La recherche étant linéaire, son temps d'exécution est proportionnel au nombre d'éléments contenus dans la structure.

L'analyse statique et dynamique de la complexité de Bind9 met en évidence une répartition hétérogène des traitements au sein du résolveur. Si la majorité des fonctions présentent une complexité relativement faible et un nombre limité d'appels, un ensemble restreint de fonctions occupe une position centrale dans l'architecture du logiciel et concentre une part importante de l'activité observée lors du traitement des requêtes DNS.

L'étude des graphes d'appels permet d'identifier ces fonctions critiques ainsi que leurs relations avec les différents sous-systèmes du résolveur. Les mesures dynamiques montrent par ailleurs que certaines fonctions voient leur nombre d'appels évoluer proportionnellement à la charge appliquée, ce qui suggère un lien direct entre leur activité, les performances du résolveur et sa consommation de ressources.

Ces résultats fournissent une meilleure compréhension du fonctionnement interne de Bind9 et permettent d'identifier les composantes susceptibles d'avoir le plus fort impact sur les performances et la consommation énergétique. Ils constituent ainsi une base de travail pertinente pour la construction de modèles visant à représenter le comportement du résolveur.

Cependant, il manque une analyse du temps d'exécution de ces fonctions, qui pourrait donner des informations supplémentaires sur la consommation énergétique du résolveur. De plus, il est probable que nous observions des différences dans le temps d'exécution de ces fonctions selon la configuration de mesure (avec ou sans cache, protocole utilisé, etc.). Il est également important de noter qu'il manque probablement des fonctions critiques dans cette analyse, pour pouvoir mesurer les fonctions manquantes, il faudrait intégrer les bibliothèques utilisées par Bind9 dans l'analyse de la complexité algorithmique. Cela permettrait d'avoir une vue plus complète de la consommation énergétique du résolveur DNS.

Les différentes campagnes de mesures réalisées ont permis de caractériser le comportement du résolveur DNS sous plusieurs angles complémentaires : utilisation du cache, consommation des ressources système, consommation énergétique et complexité algorithmique. L'analyse des résultats met en évidence l'influence de paramètres tels que le protocole utilisé, la charge appliquée ou encore l'utilisation du cache sur les performances et la consommation du système.

Au-delà des observations quantitatives, ces expérimentations ont également permis d'identifier les principaux mécanismes impliqués dans le traitement des requêtes DNS et de mieux comprendre leur contribution relative à la charge du résolveur. Les métriques collectées constituent ainsi un ensemble de données riche permettant de relier l'activité interne de Bind9 aux ressources matérielles effectivement consommées.

Ces résultats constituent le socle expérimental nécessaire à la phase de modélisation. Les analyses précédentes ont permis d'identifier les variables pertinentes, d'estimer les paramètres caractéristiques du système et de valider les hypothèses formulées sur le fonctionnement du résolveur. La section suivante s'appuie sur ces observations afin de construire et d'évaluer différents modèles capables d'estimer la consommation énergétique d'un résolveur DNS à partir de ses caractéristiques de fonctionnement.

3.5 Modélisation

L'objectif principal de ce projet est de mieux comprendre et d'étudier la consommation énergétique des infrastructures DNS, en se concentrant plus particulièrement sur le résolveur Bind9. Les campagnes de mesures présentées dans les sections précédentes ont permis de constituer un ensemble de données comprenant à la fois des mesures de consommation énergétique, des métriques système et réseau ainsi que des informations relatives à la complexité algorithmique du logiciel.

Toutefois, les mesures seules ne permettent pas d'expliquer ou de prédire directement le comportement énergétique du résolveur dans de nouvelles conditions d'utilisation. Une étape de modélisation est donc nécessaire afin de transformer ces observations expérimentales en outils capables d'estimer la consommation énergétique du système.

Deux approches complémentaires sont étudiées dans ce travail. La première repose sur des techniques d'apprentissage automatique permettant d'établir une relation entre les métriques observées et la consommation énergétique mesurée. La seconde s'appuie sur un modèle analytique fondé sur les réseaux de files d'attente, dont l'objectif est de représenter les différentes étapes du traitement d'une requête DNS et leur contribution à la consommation globale.

Ces approches répondent à des problématiques différentes mais complémentaires. Les modèles statistiques visent principalement à maximiser la précision des prédictions, tandis que les modèles analytiques cherchent à fournir une compréhension des mécanismes internes responsables de la consommation énergétique. Enfin, un troisième axe de recherche, basé sur l'analyse de la complexité algorithmique du résolveur, est actuellement à l'étude mais n'a pas encore abouti à un modèle exploitable.

3.5.1 Prédiction de la consommation énergétique du DNS

Les campagnes de mesures réalisées ont permis de collecter un grand nombre de métriques caractérisant l'activité du résolveur DNS, notamment l'utilisation du processeur, de la mémoire, du réseau, des entrées-sorties ainsi que différents indicateurs liés au fonctionnement du cache. Ces informations constituent une base de données riche permettant d'envisager une approche par apprentissage automatique.

L'objectif de cette approche est de construire un modèle capable de prédire la consommation énergétique du résolveur à partir des métriques observées. Un tel modèle présente plusieurs intérêts. D'une part, il permet d'estimer rapidement la consommation énergétique sans avoir recours à un wattmètre externe. D'autre part, il permet d'identifier les métriques les plus influentes dans la consommation observée et d'évaluer leur contribution relative.

Cette approche repose toutefois sur l'hypothèse que les métriques nécessaires à la prédiction sont disponibles au moment de l'estimation. Or, dans certains contextes de déploiement ou d'exploitation, l'accès à ces informations peut être limité. Il est donc important d'évaluer à la fois la précision du modèle obtenu et sa dépendance aux données d'entrée utilisées.

Afin d'identifier les variables les plus influentes sur la consommation énergétique du résolveur DNS, une matrice de corrélation a été calculée entre l'ensemble des métriques collectées et la consommation énergétique mesurée.

L'analyse de cette matrice met en évidence que les variables les plus fortement corrélées avec la consommation énergétique sont principalement liées à l'utilisation du processeur (CPU) ainsi qu'au débit de requêtes (QPS). Cette observation est cohérente avec le fonctionnement interne d'un résolveur DNS, dont la charge de travail est essentiellement pilotée par le nombre de requêtes traitées et le coût de traitement associé à chaque requête.

Les métriques CPU reflètent directement l'intensité du calcul nécessaire au traitement des requêtes, notamment la gestion des threads, les opérations de résolution récursive et les mécanismes de cache. De son côté, le QPS représente une mesure directe de la charge entrante, influençant linéairement ou non linéairement l'activité globale du système.

À l'inverse, certaines métriques telles que les accès disques ou certaines statistiques réseau présentent une corrélation plus faible avec la consommation énergétique dans les conditions expérimentales étudiées. Cela suggère que, pour les configurations testées, la consommation du résolveur est principalement dominée par des facteurs de calcul plutôt que par des goulots d'étranglement d'entrée/sortie.

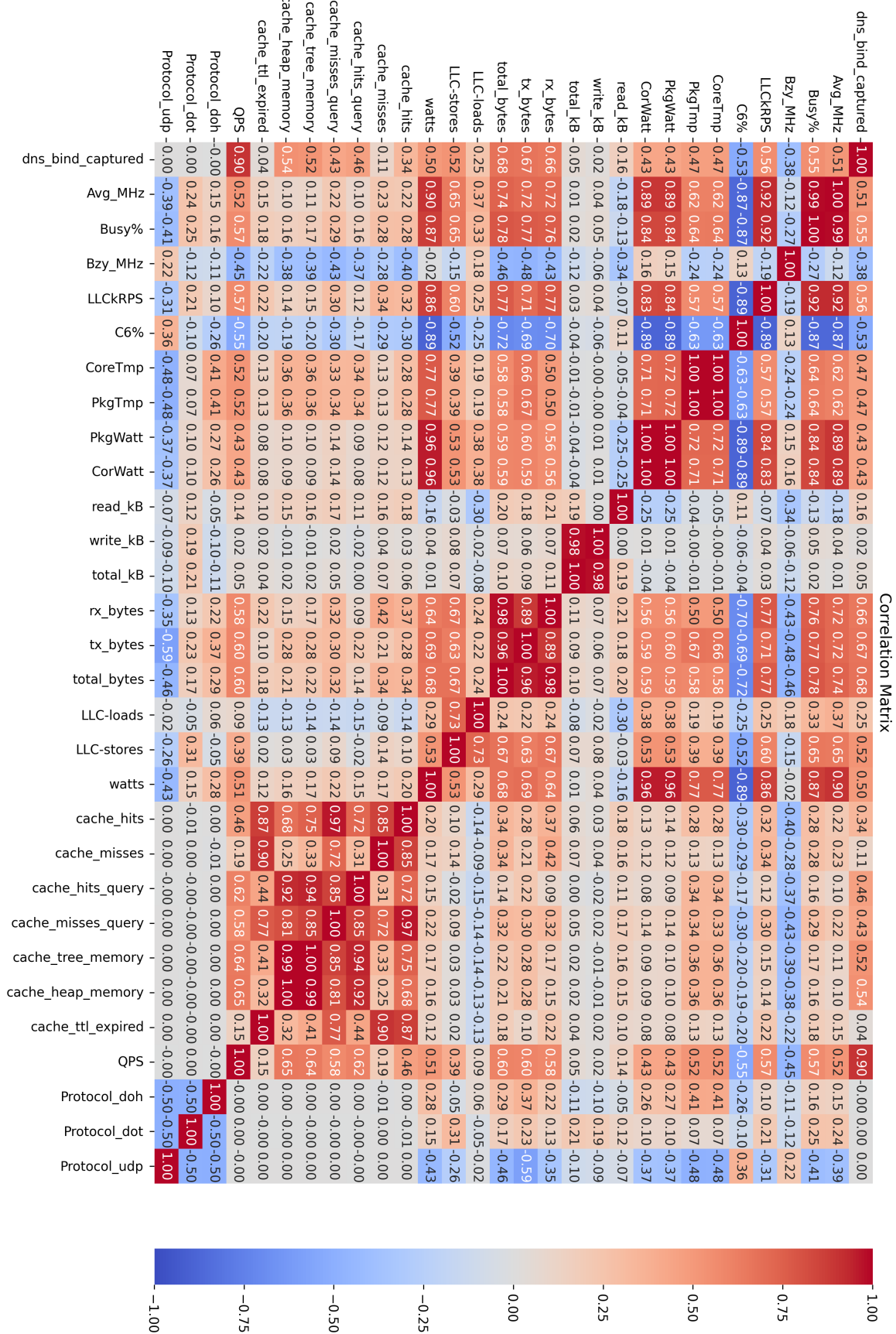


Figure 3.22 – Matrice de corrélation

Ces résultats confirment que les variables liées au CPU et au débit de requêtes constituent des indicateurs essentiels pour la modélisation de la consommation énergétique du résolveur DNS.

Cette analyse de corrélation met en évidence les variables les plus fortement liées à la consommation énergétique du résolveur DNS, en particulier celles associées à l'activité CPU et au débit de requêtes (QPS). Sur la base de ces observations, plusieurs modèles de machine learning sont ensuite entraînés afin de quantifier et exploiter ces relations pour la prédiction de la consommation énergétique.

Modèle	MSE	RMSE	MAE	MAPE (%)	Temps d'entraînement (s)
RandomForest	1.2743	1.1288	0.7988	1.2587	111.6585
XGBoost	1.2034	1.0970	0.7975	1.2632	1.6706
HistGradientBoosting	1.2305	1.1093	0.8075	1.2811	3.3806
ExtraTrees	1.3478	1.1609	0.8222	1.2965	44.4482
LinearSVR	1.7789	1.3338	0.9398	1.4847	132.775

Table 3.4 – Comparaison des performances des modèles de régression pour la prédiction de la consommation énergétique du résolveur DNS

Les résultats présentés dans le tableau 3.4 comparent plusieurs modèles de régression utilisés pour la prédiction de la consommation énergétique du résolveur DNS à partir des métriques collectées.

Globalement, les modèles basés sur des méthodes d'ensemble (*ensemble methods*) obtiennent les meilleures performances. En particulier, XGBoost atteint les erreurs les plus faibles en termes de MSE et RMSE, tout en conservant un temps d'entraînement très réduit par rapport aux autres approches. Cela en fait le modèle le plus performant en termes de compromis entre précision et coût de calcul.

Les modèles RandomForest et ExtraTrees présentent des performances légèrement inférieures, avec des erreurs plus élevées et un temps d'entraînement significativement plus important dans le cas de RandomForest. HistGradientBoosting offre des performances proches de XGBoost, mais avec un léger surcoût en erreur et en temps d'entraînement.

Enfin, le modèle linéaire LinearSVR présente les performances les moins bonnes sur l'ensemble des métriques. Cela suggère que la relation entre les variables explicatives et la consommation énergétique n'est pas linéaire, justifiant l'utilisation de modèles non linéaires plus complexes.

L'ensemble des modèles présente néanmoins des erreurs relativement faibles (MAPE autour de 1.2-1.5%), ce qui indique que les métriques collectées permettent une prédiction fiable de la consommation énergétique du résolveur DNS dans les conditions expérimentales étudiées.

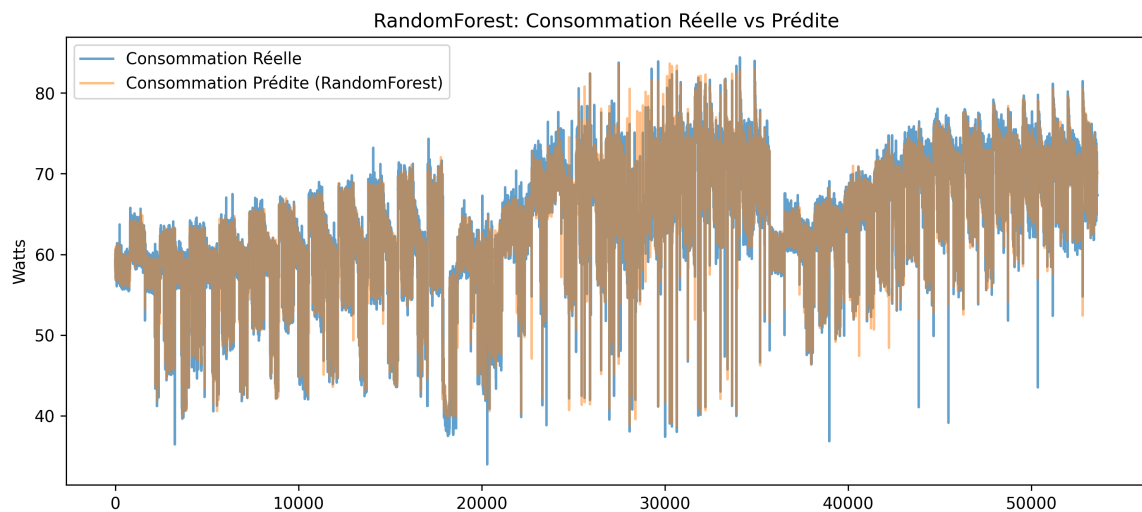


Figure 3.23 – Estimation par RandomForest et consommation énergétique réelle

Le test de prédiction est réalisé sur 20% des données, il correspond donc à plusieurs mesures de configurations différentes mise bout à bout. Ce qui explique donc ces irrégularités et cette périodicité.

L'ensemble des expérimentations réalisées dans cette section montre que la consommation énergétique du résolveur DNS peut être prédite avec une bonne précision à partir des métriques système, réseau et applicatives collectées lors des campagnes de mesures. Les différents modèles testés présentent des performances globalement proches, avec toutefois des différences notables en termes de précision et de coût d'entraînement.

Parmi les approches évaluées, XGBoost se distingue comme le modèle le plus performant. Cette supériorité s'explique par plusieurs facteurs. Tout d'abord, XGBoost repose sur une stratégie de gradient boosting optimisée, permettant de corriger itérativement les erreurs des modèles précédents et de capturer efficacement des relations non linéaires complexes entre les variables explicatives et la consommation énergétique. Ensuite, son mécanisme de régularisation intégré limite le surapprentissage, ce qui améliore sa capacité de généralisation sur des données issues de configurations variées (protocoles différents, utilisation ou non du cache, variations de charge).

De plus, XGBoost bénéficie d'optimisations d'implémentation importantes, notamment en termes de parallélisation et de gestion efficace des structures de données, ce qui explique son temps d'entraînement particulièrement faible par rapport à d'autres méthodes d'ensemble comme RandomForest ou ExtraTrees, tout en conservant une meilleure précision.

Ainsi, XGBoost constitue le meilleur compromis entre performance prédictive, robustesse et coût de calcul dans le cadre de ce problème. Ces résultats confirment l'intérêt des méthodes d'ensemble à base d'arbres pour la modélisation de systèmes complexes tels que les résolveurs DNS, où les relations entre variables sont fortement non linéaires et dépendantes du contexte d'exécution.

3.5.2 Modèle de réseau de files d'attente

Une approche permettant d'estimer la consommation énergétique d'un résolveur DNS consiste à décomposer le processus de résolution en plusieurs étapes élémentaires et à évaluer la contribution énergétique de chacune d'entre elles. Contrairement à une approche purement empirique, dépendante du matériel ou de l'implémentation utilisée, cette méthode

visé à construire un modèle plus général permettant d'estimer la consommation d'un résolveur dans différentes conditions d'utilisation.

Les réseaux de files d'attente constituent un cadre particulièrement adapté à cet objectif. Ils permettent de représenter les différentes ressources mobilisées lors du traitement d'une requête ainsi que les délais associés aux temps d'attente et aux temps de service. Chaque étape du processus de résolution peut ainsi être modélisée indépendamment avant d'être intégrée dans un modèle global.

À partir de notre compréhension du fonctionnement interne de Bind9, un modèle de réseau de files d'attente a été élaboré. Celui-ci représente les principales étapes impliquées dans le traitement d'une requête DNS, depuis l'établissement éventuel de la connexion jusqu'à la génération de la réponse. Les différentes composantes du modèle sont les suivantes :

- Établissement de la connexion TCP (pour DoT et DoH) ;
- Handshake TCP ;
- Handshake TLS et échange des certificats ;
- Résolution récursive composée de :
 - l'exécution par les threads de travail du résolveur ;
 - les échanges DNS avec les serveurs autoritatifs ;
- Recherche et insertion dans le cache DNS.

Chacune de ces composantes est représentée par une file d'attente de type M/M/C. Les paramètres associés à chaque file devront être estimés expérimentalement afin de caractériser leur comportement. Les principales caractéristiques recherchées sont :

- ρ : taux d'utilisation ou charge de la ressource ;
- S : temps moyen de service.

L'estimation de ces paramètres repose sur une étude d'ablation. Cette méthodologie consiste à ajouter progressivement les différentes étapes du processus de résolution à une configuration minimale mobilisant le moins de mécanismes possible. L'objectif est d'isoler la contribution de chaque composante à la latence observée ainsi qu'à la consommation des ressources matérielles, notamment le processeur et la mémoire.

Les protocoles de mesure envisagés pour caractériser les différentes composantes sont les suivants :

- **Threads de travail** : mesure réalisée à l'aide de requêtes UDP dont les enregistrements sont déjà présents dans le cache afin d'éliminer le coût de la résolution récursive ;
- **Recherche dans le cache** : mesure réalisée avec des requêtes UDP en faisant varier le taux de remplissage du cache afin d'évaluer l'impact de sa taille sur les temps d'accès ;
- **Résolution récursive et requêtes vers les serveurs autoritatifs** : mesure réalisée avec des requêtes UDP portant sur des noms de domaine absents du cache ;
- **Établissement de la connexion TCP** : mesure réalisée à l'aide de requêtes TCP sans couche TLS afin d'isoler le coût de la connexion ;
- **Handshake TLS et échange des certificats** : mesure réalisée en comparant les performances obtenues avec TCP seul et avec DoT ou DoH, permettant d'isoler le surcoût induit par la couche TLS.

Ce modèle repose sur notre compréhension actuelle du fonctionnement interne de Bind9. Bien qu'il ne prétende pas représenter l'intégralité des mécanismes mis en œuvre par le résolveur, il fournit un cadre méthodologique permettant d'analyser et d'estimer sa consommation énergétique.

bind9 n'utilise pas de mémoire tampon pour mettre en attente des requêtes. Les seules attentes subies par les requêtes sont dues à des délais de traitement des requêtes itératives (qu'on appelle ici récursives...). Une requête qui attend est donc *en service*. Comme

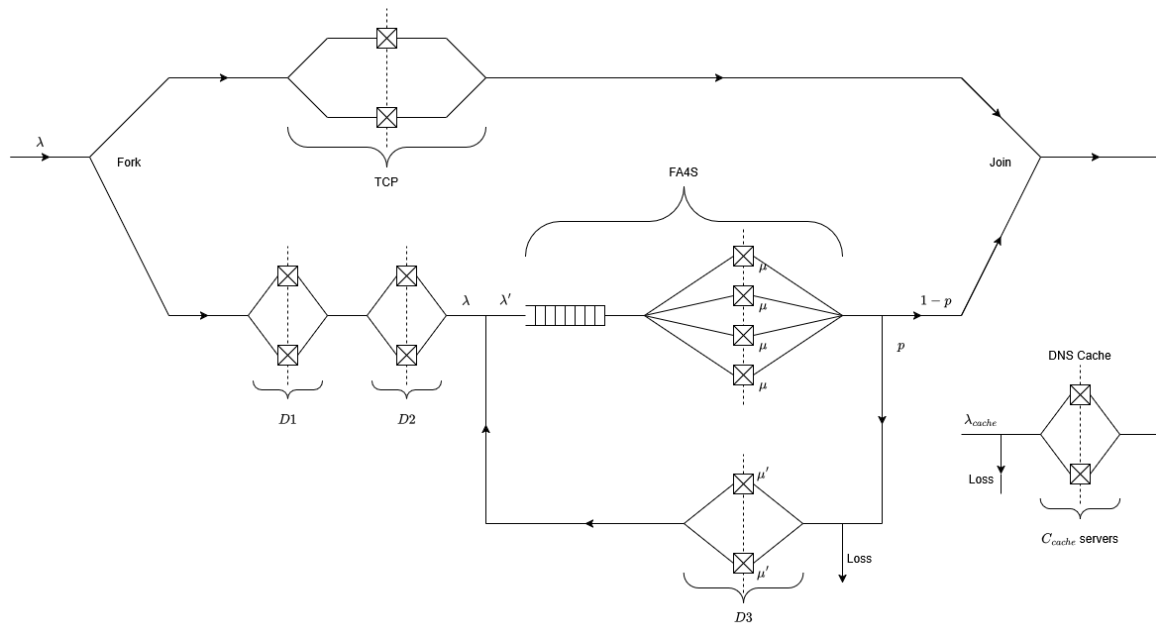


Figure 3.24 – Modèle de réseau de files d'attente représentant les principales étapes du traitement d'une requête DNS.

toutes les requêtes sont traitées en même temps, on a donc un système composé de sous-systèmes de serveurs en parallèle.

bind9 utilise certaines variables pour limiter les requêtes traitées simultanément.

Pour notre étude énergétique, les deux seules paramètres impactants sont `clients-per-query` et `recursive-clients`. La première a une valeur par défaut de 10 mais est adaptée dynamiquement. Elle est donc estimée dans la suite de ce document. La seconde est fixe, à 1000 par défaut. `clients-per-query` limite le nombre de requêtes correspondant à une même demande client. Puisqu'il y a nécessairement plusieurs noms résolus simultanément, il y a donc plusieurs blocs de `clients-per-query` serveurs. Notons qu'il faudrait revoir le débit entrant dans chacun de ces blocs et fixer les taux de services en conséquence mais comme n'intervient à ce niveau que la charge en Erlang sur chacun de ces blocs, la distinction est faite indirectement lors de l'estimation desdits taux de services.

On a déjà modélisé le système par un réseau de files d'attente 3.24. Pour proposer une solution approchée, on considère un autre réseau, présenté figure 3.25. La boucle réseau rendant compte des recherches récursives est modélisée ici par un système à perte sans boucle mais dont le débit entrant est multiplié par un facteur l correspondant à $1/(1-p)$ selon la notation du réseau initial. Le système est représenté par deux étages successifs. Le premier étage correspond au traitement (réception) initial des requêtes. Il est composé de plusieurs serveurs en parallèle modélisés par des files $M/M/C_a/C_a$. Une requête arrivant lorsque tous les serveurs du premier étage sont occupés est rejetée et produit un "serveur fail". Elle n'est donc pas envoyée vers le second étage. Le second étage est également modélisé par une file $M/M/C_r/C_r$ où C_r est fixé à 1000 dans l'estimation. La structure générale est représentée figure 3.25.

Le premier étage représente les ressources principales de traitement. Le second étage représente les requêtes récursives faites par bind9 sur d'autres serveurs.

Table 3.5 – Principaux paramètres numériques de BIND9 et valeurs par défaut

Paramètre	Valeur	Rôle
clients-per-query	10	Clients par requête récursive
max-clients-per-query	100	Limite maximale par requête récursive
recursive-clients	1000	Requêtes récursives simultanées maximales
tcp-clients	150	Connexions TCP DNS simultanées
transfers-in	10	Transferts de zones entrants simultanés
transfers-out	10	Transferts de zones sortants simultanés
transfers-per-ns	2	Transferts simultanés par serveur distant
max-cache-ttl	604800	TTL positif maximal en cache (s)
max-ncache-ttl	10800	TTL négatif maximal en cache (s)
min-cache-ttl	0	TTL minimal imposé (s)
max-recursion-depth	7	Profondeur maximale de récursion
max-recursion-queries	75	Nombre maximal de requêtes itératives
edns-udp-size	1232	Taille UDP EDNS annoncée (octets)
max-udp-size	4096	Taille UDP maximale (octets)
tcp-listen-queue	10	File d'attente TCP
heartbeat-interval	60	Intervalle heartbeat (s)
interface-interval	60	Vérification des interfaces (s)
statistics-interval	60	Intervalle statistiques (s)
lame-ttl	600	Durée de cache des réponses lame (s)
servfail-ttl	1	Durée de cache SERVFAIL (s)

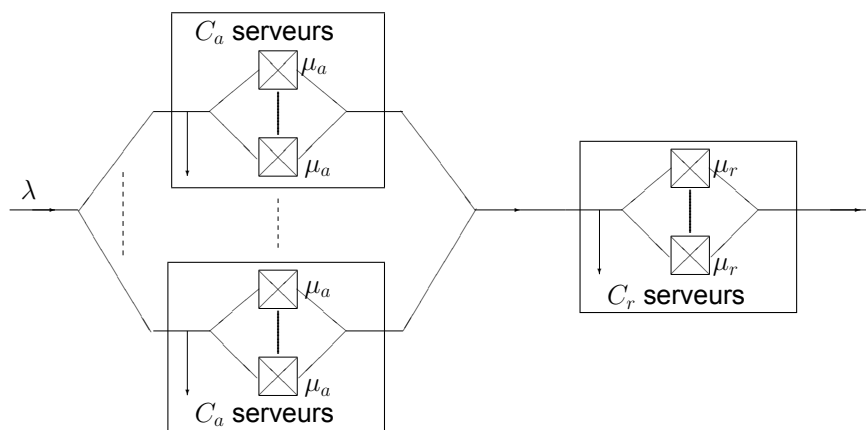


Figure 3.25 – Modèle approché de bind9

Prise en compte du cache

Cache sans rafraîchissement. Le système étudié reçoit des requêtes DNS avec un débit moyen λ (requêtes par seconde). Les arrivées sont supposées suivre un processus de Poisson. Chaque information stockée dans le cache possède une durée de vie (Time To Live, TTL) constante notée T . Une hypothèse importante du modèle est qu'une requête trouvant une information valide dans le cache ne provoque pas de rafraîchissement de cette information. Le TTL continue donc à décroître indépendamment des requêtes reçues. Sous cette hypothèse, la probabilité qu'une information ne soit pas disponible dans le cache est modélisée par :

$$P_{\text{hors cache}} = \frac{1}{1 + \lambda T}$$

On considère une donnée soumise à un processus d'arrivée de requêtes poissonnien de taux λ . Les intervalles entre deux arrivées successives sont des variables aléatoires exponentielles :

$$X \sim \text{Exp}(\lambda), \quad (3.1)$$

$$\mathbb{E}[X] = \frac{1}{\lambda}. \quad (3.2)$$

On suppose que le TTL de la donnée est constant et égal à T , et qu'une lecture de la donnée en cache ne provoque aucun rafraîchissement du TTL. Après une requête qui trouve la donnée absente, celle-ci est chargée dans le cache et y reste pendant une durée déterministe T . À l'expiration du TTL, la donnée est supprimée. Elle reste alors absente jusqu'à la prochaine requête. Soit A le temps d'attente entre l'instant d'expiration du TTL et la prochaine arrivée d'une requête. Il faut montrer que :

$$A \sim \text{Exp}(\lambda). \quad (3.3)$$

En effet, en utilisant la propriété sans mémoire de la loi exponentielle :

$$\Pr(A > t) = \Pr(X > T + t \mid X > T) \quad (3.4)$$

$$= \frac{\Pr(X > T + t)}{\Pr(X > T)} \quad (3.5)$$

$$= \frac{e^{-\lambda(T+t)}}{e^{-\lambda T}} \quad (3.6)$$

$$= e^{-\lambda t}. \quad (3.7)$$

On retrouve donc la fonction de survie d'une loi exponentielle de paramètre λ , d'où :

$$A \sim \text{Exp}(\lambda), \quad (3.8)$$

$$\mathbb{E}[A] = \frac{1}{\lambda}. \quad (3.9)$$

On définit un cycle de renouvellement comme l'intervalle entre deux chargements successifs de la donnée dans le cache. Un cycle est composé de deux phases :

- une phase de présence en cache de durée T ;
- une phase d'absence en cache de durée A .

La durée moyenne d'un cycle est donc :

$$\mathbb{E}[C] = \mathbb{E}[T + A] \quad (3.10)$$

$$= T + \frac{1}{\lambda}. \quad (3.11)$$

Par le théorème de renouvellement, la probabilité stationnaire d'observer la donnée absente du cache est égale au rapport entre la durée moyenne passée dans l'état absent et la durée moyenne d'un cycle :

$$P_{\text{miss}} = \frac{\mathbb{E}[A]}{\mathbb{E}[C]} \quad (3.12)$$

$$= \frac{\frac{1}{\lambda}}{T + \frac{1}{\lambda}} \quad (3.13)$$

$$= \frac{1}{1 + \lambda T}. \quad (3.14)$$

Ainsi :

$$P_{\text{miss}} = \frac{1}{1 + \lambda T} \quad (3.15)$$

et, par complémentarité :

$$P_{\text{hit}} = 1 - P_{\text{miss}} \quad (3.16)$$

$$= \frac{\lambda T}{1 + \lambda T}. \quad (3.17)$$

Le débit transmis au second étage est donc réduit d'un facteur :

$$\frac{1}{1 + \lambda T}$$

Cas avec rafraîchissement du cache. Une autre hypothèse possible consiste à considérer qu'une requête arrivant sur une entrée déjà présente dans le cache renouvelle son TTL. Dans ce cas, l'événement correspondant à une absence totale de requête pendant une durée T suit directement une loi de Poisson :

$$P(N(T) = 0) = e^{-\lambda T}$$

La probabilité de défaut deviendrait alors :

$$P_{\text{hors cache}} = e^{-\lambda T}$$

Le comportement du modèle serait alors différent, notamment pour les fortes charges.

Dans la suite, on utilise uniquement l'hypothèse sans rafraîchissement avec :

$$P_{\text{hors cache}} = \frac{1}{1 + \lambda T}$$

Premier étage : modèle Erlang-B

Pour un protocole donné i (UDP, DoT ou DoH), le premier étage est caractérisé par :

- $C_{a,i}$: nombre de serveurs disponibles au premier étage,
- $\mu_{a,i}$: taux de service individuel du premier étage,
- λ : débit de requêtes arrivantes.

Le premier étage est une file :

$$M/M/C_{a,i}/C_{a,i}$$

Le taux d'occupation est :

$$A_{a,i} = \frac{\lambda}{\mu_{a,i}}$$

La probabilité de rejet est donnée par la formule d'Erlang-B :

$$B(C, A) = \frac{A^C / C!}{\sum_{k=0}^C A^k / k!}$$

Le débit effectivement traité par le premier étage vaut donc :

$$\lambda_{a,i} = \lambda \left(1 - B \left(C_{a,i}, \frac{\lambda}{\mu_{a,i}} \right) \right)$$

Ce débit représente la quantité de requêtes effectivement traitées par les ressources du premier étage, par unité de temps.

Second étage

Les requêtes traitées par le premier étage sont envoyées vers le second. Le modèle introduit un facteur l représentant l'amplification du trafic entre les deux étages. Avec la modélisation du cache présentée précédemment, le débit arrivant au second étage est :

$$\lambda_{r,i} = \frac{l\lambda_{a,i}}{1 + \lambda T}$$

Le second étage est également modélisé par une file $M/M/C_r/C_r$ avec $C_r = 1000$ et un taux de service commun μ_r . La probabilité de rejet du second étage vaut :

$$B_r = B \left(C_r, \frac{\lambda_{r,i}}{\mu_r} \right).$$

Modèle énergétique

Le modèle énergétique utilisé ne cherche pas à calculer une énergie accumulée mais une puissance instantanée. Ainsi, le coût énergétique est supposé proportionnel au débit de requêtes traité par chaque étage. Un modèle proportionnel au nombre moyen de clients dans une file correspondrait à une quantité d'énergie (en Joules), alors que la grandeur mesurée ici est une puissance (Watt). Le coût du premier étage est donc :

$$P_{a,i} = a_{a,i} \lambda_{a,i}$$

Le coût du second étage est :

$$P_r = a_r \lambda_{r,i} (1 - B_r)$$

Le modèle complet devient alors :

$$P_i(\lambda) = a_{a,i} \lambda \left(1 - B \left(C_{a,i}, \frac{\lambda}{\mu_{a,i}} \right) \right) + a_r \frac{l \lambda \left(1 - B \left(C_{a,i}, \frac{\lambda}{\mu_{a,i}} \right) \right)}{1 + \lambda T} \times \left[1 - B \left(C_r, \frac{l \lambda (1 - B(C_{a,i}, \lambda / \mu_{a,i}))}{\mu_r (1 + \lambda T)} \right) \right] + b. \quad (3.18)$$

Cette expression est utilisée pour chacun des trois protocoles :

$$i \in \{UDP, DoT, DoH\}$$

Les paramètres C_r , μ_r , l , T , a_r et b sont communs, alors que $a_{a,i}$, $C_{a,i}$ et $\mu_{a,i}$ dépendent du protocole.

3.5.3 Méthode d'estimation des paramètres

Les paramètres sont obtenus par minimisation d'une erreur quadratique entre les mesures expérimentales et le modèle. La fonction objectif utilisée est :

$$J = \frac{1}{6N} \sum_{j=1}^N \sum_i (P_i(q_j) - P_{i,j}^{mes})^2$$

où :

- N est le nombre de points expérimentaux,
- q_j est le débit mesuré,
- $P_{i,j}^{mes}$ est la puissance mesurée,
- $P_i(q_j)$ est la prédiction du modèle.

L'optimisation est réalisée avec l'algorithme de Nelder-Mead. Cette méthode ne nécessite pas de calcul de gradient et est adaptée aux modèles comportant des paramètres discrets comme le nombre de serveurs C_a .

Les paramètres obtenus sont les suivants (cf. tables 3.6 et 3.7). L'erreur finale obtenue est :

$$J = 5.641782617397 \times 10^{-2}$$

3.5.4 Validation expérimentale

Validation à faible charge

Le modèle estimé est comparé aux mesures expérimentales fournies pour les trois protocoles DNS *UDP*, *DoT*, *DoH*. Les données expérimentales contiennent pour chaque débit q deux valeurs :

- une mesure sans utilisation du cache,

Paramètre	Valeur	Description
l	1.8092142062	amplification trafic
T	463.8215739360	TTL cache
b	31.4794837097	offset énergétique
a_r	0.0154433308	coût second étage
μ_r	55.6251558161	service second étage
C_r	1000	fixé

Table 3.6 – Paramètres communs estimés

Protocole	a_a	C_a	μ_a
UDP	0.0128657655	13	44.3397529922
DoT	0.0225331164	20	22.8950701918
DoH	0.0204781190	15	33.5475106740

Table 3.7 – Paramètres spécifiques aux premiers étages

— une mesure avec cache.

Les paramètres estimés précédemment sont utilisés sans modification. La première comparaison est réalisée sur l'intervalle correspondant aux mesures disponibles [$0 \leq q \leq 550$]. Elle est donnée figure 3.26.

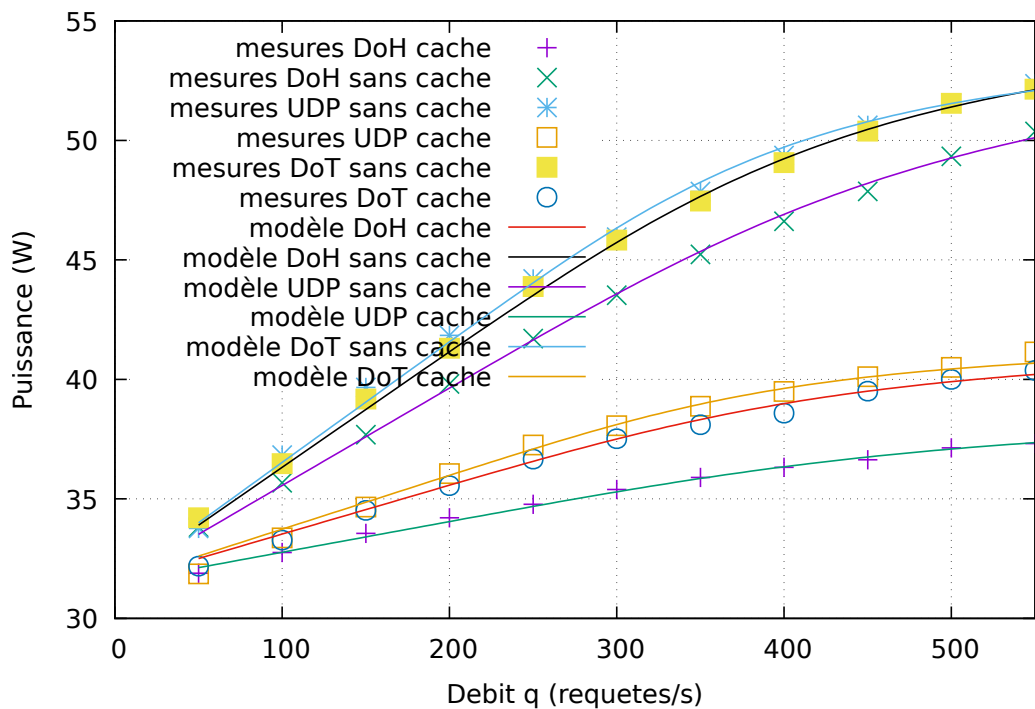


Figure 3.26 – Comparaison modèle-mesures dans la plage expérimentale.

Extrapolation à forte charge

Afin d'étudier le comportement du modèle lorsque le système est fortement chargé, une seconde représentation est effectuée jusqu'à [$0 \leq q \leq 7000$]. Le même jeu de paramètres

est conservé. Elle est donnée figure 3.27.

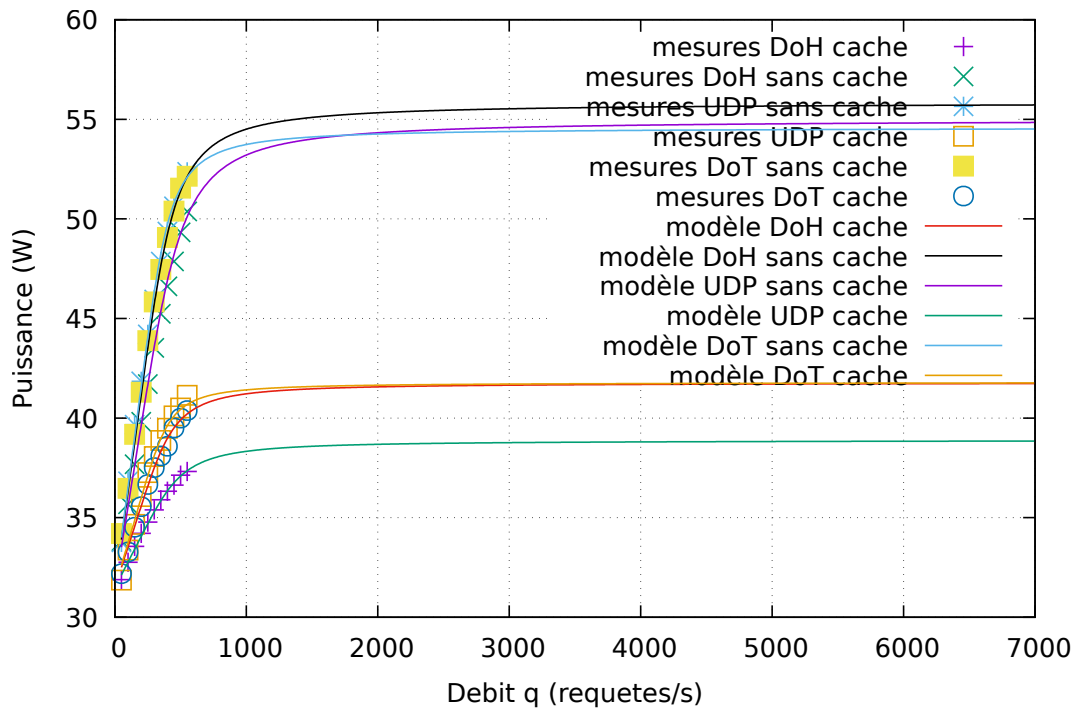


Figure 3.27 – Extrapolation du modèle jusqu'à une charge élevée.

Influence de l'hypothèse de cache

Le modèle utilisé repose sur :

$$P_{\text{hors cache}} = \frac{1}{1 + \lambda T}$$

Cette hypothèse correspond à un cache avec TTL fixe et sans rafraîchissement lors d'un accès. Une autre possibilité consisterait à considérer qu'une arrivée de requête renouvelle systématiquement la durée de vie de l'entrée si elle est déjà présente. Le taux de défaut deviendrait alors :

$$P_{\text{hors cache}} = e^{-\lambda T}$$

Dans ce cas, le débit transmis au second étage serait :

$$\lambda_r = l\lambda_a e^{-\lambda T}$$

et non plus :

$$\lambda_r = \frac{l\lambda_a}{1 + \lambda T}$$

Les courbes obtenues avec cette seconde hypothèse nécessiteraient donc une nouvelle estimation complète des paramètres. Les valeurs :

$$l, T, a_a, a_r, \mu_a, \mu_r$$

ne peuvent pas être conservées car elles rendent compte de la dynamique spécifique du cache sans rafraîchissement à chaque arrivée.

Limites du modèle

Deux phénomènes physiques importants ne sont actuellement pas pris en compte.

Coût énergétique du cache. Le modèle suppose que le cache est gratuit. La puissance totale est donc écrite :

$$P = P_{\text{traitement}}$$

En réalité, un cache possède un coût propre :

- consommation mémoire,
- accès mémoire,
- gestion des entrées,
- expiration des TTL,
- synchronisation.

Une extension possible serait d'ajouter un terme :

$$P_{\text{cache}} = a_c N_c$$

où N_c représente le nombre moyen d'entrées présentes dans le cache. Le modèle deviendrait alors :

$$P = P_{\text{traitement}} + P_{\text{cache}}$$

Arrivée permanente de nouveaux noms de domaine. Le modèle actuel suppose implicitement que les requêtes concernent un ensemble stable de noms déjà connus. Or un système DNS réel reçoit continuellement de nouveaux noms :

$$\lambda = \lambda_{\text{connus}} + \lambda_{\text{nouveaux}}$$

Les nouveaux noms ont une probabilité élevée de provoquer un défaut de cache et modifient donc la dynamique du second étage. Une modélisation plus complète devrait introduire :

- une distribution des popularités des noms,
- une dynamique de remplissage du cache,
- un renouvellement des objets stockés.

Une loi de type Zipf pourrait par exemple être utilisée pour représenter la distribution des fréquences de requêtes.

3.5.5 Conclusion

Le modèle proposé permet de relier la consommation énergétique d'un système DNS à sa charge, à ses capacités de traitement et à son mécanisme de cache. L'utilisation de files Erlang-B permet de représenter les phénomènes de saturation et de rejet. Les paramètres obtenus reproduisent correctement les mesures expérimentales dans la plage étudiée. Des extensions futures devront cependant intégrer le coût énergétique du cache et la dynamique réelle d'apparition des nouveaux noms de domaine.

Conclusion

Ce stage s'inscrit dans le cadre du projet DECODES, dont l'objectif est de mieux comprendre et de modéliser la consommation énergétique des infrastructures DNS. Plus précisément, ce travail s'est concentré sur l'étude du résolveur Bind9 à travers la mise en place d'un environnement expérimental, la réalisation de campagnes de mesures et le développement de différentes approches de modélisation. Au-delà du cas particulier du DNS, cette étude constitue une première étape vers une problématique plus générale : l'estimation de la consommation énergétique des systèmes distribués.

Les travaux réalisés ont permis de proposer une méthodologie complète de mesure de la consommation énergétique d'un résolveur DNS, d'analyser l'impact de différents protocoles et mécanismes tels que le cache, et d'étudier plusieurs approches de modélisation. Les modèles de machine learning ont démontré qu'il est possible de prédire avec précision la consommation énergétique à partir de métriques système, tandis que le modèle de réseau de files d'attente offre une approche plus interprétable permettant d'analyser la contribution des différentes étapes du traitement d'une requête. En parallèle, l'analyse de la complexité algorithmique de Bind9 apporte un point de vue complémentaire en reliant le comportement du logiciel à sa consommation, tout en s'affranchissant largement des caractéristiques matérielles de la plateforme d'exécution.

Ces travaux ouvrent également plusieurs perspectives. Les différences de consommation observées entre les mesures instrumentées et les mesures sans collecte de métriques système méritent d'être étudiées plus en profondeur, car elles mettent en évidence le coût non négligeable de l'instrumentation. Cette problématique est particulièrement importante dans la perspective d'un déploiement en environnement de production, notamment au sein des centres de données, où la mise en œuvre de mesures énergétiques externes est complexe. Par ailleurs, la construction de modèles représentatifs de résolveurs DNS en production nécessitera un accès à des données réelles, ce qui soulève des enjeux de collaboration avec les opérateurs de centres de données et de partage des données d'exploitation.

Sur le plan personnel, ce stage a constitué une expérience particulièrement enrichissante. Il m'a permis de découvrir le fonctionnement de la recherche académique, de participer à un projet scientifique de longue durée et d'acquérir une méthodologie rigoureuse pour la conception d'expérimentations, l'analyse critique des résultats et la modélisation de systèmes complexes. Cette expérience a renforcé mon intérêt pour la recherche et je suis particulièrement enthousiaste à l'idée de poursuivre ces travaux dans le cadre d'une thèse.

Enfin, je tiens à remercier chaleureusement mes encadrants pour leur disponibilité, leurs conseils et leur accompagnement tout au long de ce stage. Je remercie également l'ensemble doctorants et autres stagiaires présents à Télécom Paris pour les discussions enrichissantes, les échanges d'idées et les bons moments partagés.

Bibliographie

- [1] Nicolas Renout et al. « Analysing the Energy Consumption of a DNS Resolver ». In : *2025 9th Network Traffic Measurement and Analysis Conference (TMA)*. 2025. doi : 10.23919/TMA66427.2025.11097017.
- [2] Capucine Barré et al. « Quantifying CPU Configuration Effects on DNS Resolver Power Consumption ». In : *DECODES Project Report / Technical Paper* (2026).
- [3] Jayant Baliga, Kerry Hinton et Rodney S. Tucker. « Energy Consumption of the Internet ». In : *COIN-ACOFT 20011 - Joint International Conference on the Optical Internet and the 32nd Australian Conference on Optical Fibre Technology*. 2007. doi : 10.1109/COINACOFT.2007.4519173.
- [4] Dominique Dudkowski et Konstantinos Samdanis. « Energy consumption monitoring techniques in communication networks ». In : *2012 IEEE Consumer Communications and Networking Conference (CCNC)*. 2012. doi : 10.1109/CCNC.2012.6181048.
- [5] John C. McCullough et Yuvraj Agarwal. « Evaluating the Effectiveness of Model-Based Power Characterization ». In : *2011 USENIX Annual Technical Conference (USENIX ATC 11)*. Portland, OR : USENIX Association, juin 2011. Disponible sur : <https://www.usenix.org/conference/usenixatc11/evaluating-effectiveness-model-based-power-characterization-0>.
- [6] Aniket Babuta et al. « Power and energy measurement devices : A review, comparison, discussion, and the future of research ». In : *Measurement* 172 (2021), p. 108961. issn : 0263-2241. doi : <https://doi.org/10.1016/j.measurement.2020.108961>. Disponible sur : <https://www.sciencedirect.com/science/article/pii/S0263224120314354>.
- [7] Mathilde Jay et al. « An experimental comparison of software-based power meters : focus on CPU and GPU ». In : *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. 2023. doi : 10.1109/CCGrid57682.2023.00020.
- [8] Albert Hankel et al. « Measuring software energy efficiency : Presenting a methodology and case study on DNS resolvers ». In : *2016 18th Mediterranean Electrotechnical Conference (MELECON)*. 2016. doi : 10.1109/MELCON.2016.7495390.
- [9] Etienne LE Louet et al. « Effects of secured DNS transport on resolver performance ». In : *2023 IEEE Symposium on Computers and Communications (ISCC)*. 2023. doi : 10.1109/ISCC58397.2023.10217887.
- [10] Lars Karlslund. topdomains. GitHub, 2024. Disponible sur : <https://github.com/lkarlslund/topdomains>.
- [11] Yoctopuce. last access june 2026. Disponible sur : <https://www.yoctopuce.com/FR/products/capteurs-electriques-usb/yocto-watt>.
- [12] Sarah Kasdi. « How to design an experiment to measure and compare the electricity consumption of a server machine across network topologies ? » In : (2025).

Annexes

Détail des métriques collectées

Nouvelles métriques

Le cache sera surveillé à l'aide de 6 variables disponibles via `rndc`, un module utilisé pour visualiser les métriques de Bind9.

Les nouvelles métriques sont :

Accès au cache

Le nombre total de requêtes DNS **servies directement depuis le cache**.

Cela inclut les réponses positives (par exemple, les enregistrements A, AAAA, MX) qui ont été précédemment mises en cache.

Unité : Compteur (entier non signé).

Échecs de cache

Le nombre total de requêtes DNS **non servies depuis le cache**, nécessitant une résolution externe (par exemple, en interrogeant un serveur DNS autoritatif ou récursif).

Unité : Compteur (entier non signé).

Accès au cache (à partir des requêtes)

Le nombre de **réponses en cache** pour les requêtes entrantes.

Contrairement aux accès au cache, cette métrique se concentre sur les **requêtes initiales** servies depuis le cache.

Unité : Compteur (entier non signé).

Échecs de cache (à partir des requêtes)

Le nombre de requêtes entrantes **non résolues par le cache**, nécessitant une résolution externe.

Complémentaire aux accès au cache (à partir des requêtes) pour évaluer l'efficacité du cache sur les requêtes entrantes.

Unité : Compteur (entier non signé).

Mémoire de l'arbre de cache

Mémoire **utilisée par la structure arborescente du cache** (en octets).

Représente la mémoire allouée pour organiser les entrées du cache dans une structure arborescente (optimisée pour des recherches rapides).

CPU Unité : Octets.

Mémoire du tas de cache

Mémoire **utilisée par le tas du cache** (en octets).

Représente la mémoire allouée dynamiquement pour les entrées du cache non organisées dans l'arbre.

Unité : Octets.

Entrées de cache supprimées en raison de l'épuisement de la mémoire

Le nombre d'**entrées de cache supprimées** en raison de l'**épuisement de la mémoire**.

Indique que le cache a atteint sa limite de mémoire allouée (`max-cache-size`) et a dû supprimer des entrées pour libérer de l'espace.

Unité : Compteur (entier non signé).

Note : Une valeur > 0 suggère que `max-cache-size` devrait être augmentée.

Entrées de cache supprimées en raison de l'expiration du TTL

Le nombre d'**entrées de cache supprimées** car leur **TTL a expiré**.

Représente le nombre d'enregistrements DNS automatiquement supprimés du cache après avoir atteint leur durée de vie maximale.

Unité : Compteur (entier non signé).

`Time_Of_Day_Seconds` : Wall-clock timestamp of the sample (seconds since midnight). `Avg_MHz` : Average CPU frequency including idle time. `Busy%` : Percentage of time the CPU cores were active (not idle). `Bzy_MHz` : Average CPU frequency while cores were busy. `C6%` : Percentage of time cores spent in deep sleep (C6). `CoreTmp` : Average CPU core temperature (°C). `PkgTmp` : Temperature of the full CPU package (°C). `PkgWatt` : Total CPU package power consumption (watts). `CorWatt` : Power consumed by CPU cores only (watts). `LLCkRPS` : Total LLC accesses per second, in thousands (loads + stores). `LLC%hit` : Percentage of LLC accesses that hit the cache instead of going to DRAM.

Types de requêtes DNS

Principaux types de requêtes et codes de réponse DNS

Le tableau 3.8 présente les principaux types d'enregistrements DNS ainsi que les codes de réponse étudiés dans ce travail.

Signification des symboles fréquemment rencontrés dans les traces DNS

Avec les requêtes de résolution DNS en UDP, les types ont également des symboles associés, les types de symboles sont décrits dans le tableau 3.9.

Table 3.8 – Types d'enregistrements et codes de réponse DNS

Type	Description
A	Associe un nom de domaine à une adresse IPv4.
AAAA	Associe un nom de domaine à une adresse IPv6.
NS	Désigne les serveurs DNS faisant autorité pour une zone.
CNAME	Définit un alias vers un autre nom de domaine.
DS	Contient l'empreinte cryptographique d'une clé DNSSEC de la zone enfant afin d'assurer la chaîne de confiance.
DNSKEY	Contient les clés publiques utilisées pour la validation des signatures DNSSEC.
RRSIG	Contient les signatures cryptographiques associées aux enregistrements protégés par DNSSEC.
NXDOMAIN	Code de réponse indiquant que le nom de domaine demandé n'existe pas.
SERVFAIL	Code de réponse signalant une erreur interne du serveur lors du traitement de la requête.
REFUSED	Code de réponse indiquant que le serveur refuse de traiter la requête conformément à sa politique de sécurité ou de configuration.
FORMERR	Code de réponse indiquant que la requête DNS reçue est mal formée.
BADCOOKIE	Code de réponse indiquant que le cookie DNS fourni par le client est invalide ou expiré.

Table 3.9 – Signification des symboles observés dans les traces DNS

Symbole	Description
?	Indique une requête DNS adressée à un serveur.
*	Représente un caractère générique (<i>wildcard</i>) pouvant correspondre à plusieurs noms de domaine.
-	Indique généralement une valeur absente, inconnue ou non applicable.
	Utilisé comme séparateur de champs dans certains formats de traces ou de journaux.

Sujet de stage et description du travail réalisé

Le Domain Name System (DNS) est une infrastructure essentielle d'Internet dont la consommation énergétique reste encore peu étudiée. Ce stage, réalisé dans le cadre du projet DECODES à Télécom SudParis, vise à mieux comprendre et modéliser la consommation énergétique d'un résolveur DNS, en se concentrant sur l'implémentation Bind9.

Pour atteindre cet objectif, un environnement expérimental a été conçu afin de mesurer la consommation énergétique du résolveur sous différentes configurations (protocoles, utilisation du cache et charge). Les données collectées ont permis d'analyser l'influence de ces paramètres sur la consommation énergétique, d'étudier la complexité algorithmique de Bind9 et de développer deux approches de modélisation : un modèle de machine learning basé sur des métriques système et un modèle analytique fondé sur les réseaux de files d'attente. En parallèle, une analyse de la complexité algorithmique de Bind9 a été réalisée afin de mieux comprendre les facteurs logiciels influençant cette consommation.

Les résultats montrent qu'il est possible de prédire avec précision la consommation énergétique du résolveur à partir des métriques collectées, tout en mettant en évidence les principales composantes responsables de cette consommation. Ces travaux constituent une première étape vers la modélisation énergétique de systèmes distribués plus complexes.

Mots-clés : DNS, Bind9, consommation énergétique, modélisation, machine learning, réseaux de files d'attente, complexité algorithmique.

Télécom SudParis

19 Place Marguerite Perey
91120 Palaiseau

www.telecom-sudparis.eu